

Dynamic Decentralized Functional Encryptions from Pairings in the Standard Model

Duy Nguyen^[0009–0002–7892–9146]

Telecom Paris, Institut Polytechnique de Paris, France
dinh.nguyen@telecom-paris.fr

Abstract. Dynamic Decentralized Functional Encryption (DDFE), introduced by Chotard *et al.* (CRYPTO'20), represents a robust generalization of (Multi-Client) Functional Encryption. It allows users to dynamically join and contribute private inputs to individually controlled joint functions without requiring a trusted authority.

Recently, Shi and Vanjani (PKC'23) proposed the first Multi-Client Functional Encryption scheme for function-hiding inner products (FH-IP) without relying on random oracles. Unfortunately, their construction still requires a trusted key authority, leaving open the question of whether a full-fledged FH-IP-DDFE can exist in the standard model.

In this work, we answer this question affirmatively by introducing Updatable Pseudorandom Zero Sharing, a novel concept that provides both the critical functionality and security properties needed to construct secure DDFE schemes in the standard model.

Our second contribution is a novel proof strategy, which preserves adaptive security when transforming any functional encryption scheme for FH-IP into FH-IP-DDFE. Together, these two techniques enable a modular construction of FH-IP-DDFE that is secure against adaptive message and key queries in the standard model.

Additionally, our pseudorandom zero-sharing scheme is highly versatile, enabling the first DDFE for attribute-weighted sums in the standard model, complementing the recent ROM-based construction by Agrawal *et al.* (CRYPTO'23).

Keywords: dynamic decentralized, functional encryption, pairing, standard model

1 Introduction

1.1 Dynamic Decentralized Functional Encryption

Functional Encryption. Functional Encryption (FE) is a robust cryptographic paradigm introduced by Sahai and Waters [SW05, BSW11]. It addresses the all-or-nothing limitations of standard public-key cryptosystems by offering fine-grained control over access to encrypted data through functional decryption keys. More precisely, each ciphertext ct_x encrypts a specific value x , and each decryption key dk_f encapsulates a function f . When a receiver uses dk_f to decrypt ct_x , what it can learn is only the result $f(x)$, and nothing more about x .

Abdalla *et al.* [ABDP15] addressed the challenge of constructing FE based on standard assumptions for restricted, yet expressive, classes of computational functions. In their work, they presented FE constructions for inner products (IPFE), where each private input is represented as a vector $\mathbf{x}$, each function is represented by a vector $\mathbf{y}$, and decryption yields $\mathbf{x}^\top \cdot \mathbf{y}$.

Within the realm of practical FE based on standard assumptions for computation over encrypted data, starting from [ABDP15], significant improvements have been made in terms of security [BJK15, ALS16, ALMT20], functionality [BCFG17, AGW20, ACGU20], efficiency [CLT18, TT18, DP19, MKMS22] and support for multi-user scenarios [ACF⁺18, CDG⁺18, CDSG⁺20, ZLZ⁺24].

Functional Encryption for Multiple Users. The concepts of Multi-Input Functional Encryption (MIFE) and Multi-Client Functional Encryption (MCFE) were introduced in [GGG⁺14, GKL⁺13]. These concepts generalize FE to enable multiple clients to independently contribute their encrypted individual inputs to the computation of joint functions. Concretely, with the help of possibly a trusted authority, each slot owner receives a private encryption key sk_i and encrypts an input x_i under some label ℓ as $\text{ct}_{x_i, \ell}$. By collecting ciphertexts of all slot owners $(\text{ct}_{x_i, \ell})_i$ under a same label and a functional decryption key dk_f generated from the authority's master secret key msk , a receiver can obtain no more information than $f((x_i)_i)$.

In the multi-client setting, each slot owner is assumed to be an independent client: the security guarantees that only ciphertexts under a same label can be decrypted together, and the input privacy of honest clients holds even when a subset of clients is corrupted. In the multi-input setting, all labels are assumed to be the same, all slot owners are originally assumed to be the same user (yet each input can be sent independently instead of simultaneously as in single-input FE), and then no corruption is considered.

Therefore, any MCFE designed for a specific functionality automatically implies a MIFE for the same functionality by using a fixed label for all encryptions. Conversely, an MIFE designed for general functions directly implies an MCFE for general functions, as the label can be contained in every plaintext, and the function can verify that every slot uses the same label. However, when considering MIFE schemes for only restricted classes of functions in the literature, this equivalence does not hold.

Since their introduction, the research line on practical MIFE/MCFE constructions has witnessed a dynamic array of works seeking developments in the aspects of security [CDG⁺18, LT19, AGT22, SV23], functionality [CDSG⁺20, ACGU20, AGT21a, NPP22, ATY23], and in the relation with single-input FE [ABG19].

Dynamic Decentralized Functional Encryption. Standard MCFE models often encounter the key escrow problem, stemming from the reliance on a trusted authority. To address this challenge, Chotard *et al.* introduced the concept of decentralized MCFE (DMCFE) [CDG⁺18] and later Dynamic Decentralized Functional Encryption (DDFE) [CDSG⁺20]. In DMCFE, the need for a trusted authority is entirely removed, granting each client complete control over their encrypted data and the generation of functional decryption keys. DDFE is an extended notion of DMCFE by preserving all decentralized features while enabling the join of new clients at various stages during the lifetime of the system through a non-interactive setup.

The development of DMCFE/DDFE since the introduction of the first scheme for inner products [CDG⁺18] has seen significant progress on various aspects. These include a variety of functionalities, as seen in constructions for sums, inner products, and all-or-nothing message encapsulations in [CDSG⁺20] and attribute-weighted sums in [ATY23]. Efforts to enhance security have led to DMCFE constructions for function-hiding inner products [AGT21b, NPS24], for lattice-based assumptions in the standard model [LT19], and for optimal security notions [NPP23b]. The relations between DMCFE and other FE notions have been established in these works [ABG19, ABKW19, AGT21b]. Practical features like verifiability [NPP23a] and robustness [LWG⁺23] have also been integrated, enhancing the applicability of DMCFE in various real-world scenarios.

Multi-Party Functional Encryption. In addition to the above notions of functional encryption, there are other concepts designed for various multi-user settings. These include those supporting distributed keys [MJ18, Cha07, LW11, BCFG17], as well as those supporting both distributed ciphertexts and keys [ACF⁺20]. All these notions fall under the umbrella of Multi-Party Functional Encryption [AGT21b]. However, in this paper, our focus will be exclusively on DDFE, which supports both distributed ciphertexts and keys and stands out as the most generalized notion within Multi-Party Functional Encryption.

Open Problems in DDFE. To the best of our knowledge, DDFE constructions based on standard assumptions remain limited to the classes of function-hiding inner products (FH-IP) and function-revealing attribute-weighted sums (AWS), which strictly capture the class of function-revealing inner products. These functionalities can be described more precisely as follows:

- In DDFE for FH-IP: each client, identified by a public key $\mathbf{pk} \in \mathcal{PK}$, uses its own corresponding secret key $\mathbf{sk}_{\mathbf{pk}}$ to encrypt its private input $\mathbf{x}_{\mathbf{pk}}$ under a public message label ℓ_M and for a public user list $\mathcal{U}_M$. Similarly, each client uses $\mathbf{sk}_{\mathbf{pk}}$ to generate a decryption key for its private vector $\mathbf{y}_{\mathbf{pk}}$ under a public key label ℓ_K and for a public user list $\mathcal{U}_K$. The labels ℓ_M and ℓ_K impose constraints on which messages and functions can be aggregated, respectively, while the lists $\mathcal{U}_M$ and $\mathcal{U}_K$ specifies the sets of parties whose ciphertexts and decryption keys can be combined, respectively. The FH-IP functionality verifies the conditions $[\mathcal{U}_M = \mathcal{U}_K]$, $[(\ell_M, \mathcal{U}_M)$ is consistent across all ciphertexts] and $[(\ell_K, \mathcal{U}_K)$ is consistent across all decryption keys] during decryption. If these conditions are met, the FH-IP functionality outputs

$$\sum_{\mathbf{pk} \in \mathcal{U}_K} \mathbf{x}_{\mathbf{pk}}^\top \cdot \mathbf{y}_{\mathbf{pk}};$$

otherwise, it outputs nothing. The function-hiding security ensures that no additional information on individual $\mathbf{x}_{\text{pk}}$ and $\mathbf{y}_{\text{pk}}$ is revealed.

- In DDFE for AWS: each client pk chooses a user list $\mathcal{U}_M$, a label ℓ to encrypt its AWS inputs $\{\mathbf{z}_{\text{pk},j}\}_{j \in [N_{\text{pk}}]}$ which are private and $\{\mathbf{x}_{\text{pk},j}\}_{j \in [N_{\text{pk}}]}$ which are public. For key generation, each client pk chooses a set of users $\mathcal{U}_K$ and a list of arithmetic branching programs (see [TW14, AGW20]) $\mathbf{f} = \{f_{\text{pk}}\}_{\text{pk} \in \mathcal{U}_K}$. The functionality verifies the conditions $[\mathcal{U}_M = \mathcal{U}_K]$, $[(\ell, \mathcal{U}_M)$ is consistent across all ciphertexts] and $[\mathbf{f}$ is consistent across all decryption keys]. If these conditions are met, the AWS functionality outputs

$$\sum_{\text{pk} \in \mathcal{U}_K} \sum_{j \in [N_{\text{pk}}]} f_{\text{pk}}(\mathbf{x}_{\text{pk},j})^\top \cdot \mathbf{z}_{\text{pk},j};$$

otherwise, it outputs nothing. The security ensures that no additional information on individual $\{\mathbf{z}_{\text{pk},j}\}_{j \in [N_{\text{pk}}]}$ is revealed.

It is important to highlight that all state-of-the-art DDFE/DMCFE constructions [AGT21b, ATY23, NPS24] for the above functionalities are pairing-based and rely on random oracles for their security proofs.

On the other hand, in the development of (Decentralized) MCFE for (function-hiding) inner products, several works [LT19, ABG19, SV23] have focused on achieving standard-model security for their respective (Decentralized) MCFE schemes. Our work contributes to this ongoing research by addressing the first question:

Do there exist any DDFE constructions for function-hiding inner products and for attribute-weighted sums that are secure under standard assumptions in the standard model?

All existing FH-IPFE schemes rely on pairings, while lattice-based constructions have been proven impossible under common approaches [Üna20], therefore the second question we explore is:

Does the adaptive security of any pairing-based FE for function-hiding inner products imply the adaptive security¹ of DDFE for the same functionality?

This work provides affirmative answers to both questions, resulting in a modular construction of adaptively secure FH-IP-DDFE and a modular construction of selectively secure AWS-DDFE, both within the standard model.

1.2 Our Contributions

The contributions can be listed as follows:

- **Novel Pseudorandom Zero Sharing:** We introduce a new concept called *Updatable Pseudorandom Zero Sharing* (UZS). Compared to standard pseudorandom zero sharing, this notion simultaneously offers the following properties:
 1. it enables the local update from a zero share into a new one, and this updating algorithm can support the *bilinear update* property, which ensures the compatibility of the algorithm with operations allowed in pairing groups;
 2. its security additionally guarantees the pseudorandomness for the updated shares even if their corresponding non-updated shares are revealed; this security holds against an unbounded subset of corrupted parties;
 We demonstrate the feasibility of this notion by providing a construction in DDH groups, which is notably secure in the standard model.
- **Dynamic Decentralized Functional Encryptions Without RO:** Using UZS as a building block, we provide the first DDFE constructions for function-hiding inner products and attribute-weighted sums that are secure in the standard model. Comparative details with related works are provided in Figure 1.
- **Injection Lemma for Adaptive Security of FH-IP-DDFE:** As another independent contribution, in the security proof, we introduce an essential lemma that enables achieving adaptive security for FH-IP-DDFE from the adaptive security of *any* FH-IPFE scheme. The novel proof helps modularize the DDFE construction and establishes a previously unknown security relation between these two primitives.

¹ Here, adaptive security for DDFE refers to security under adaptively chosen messages and functions, similar to that of single-input FE.

Scheme	Function class	Function hiding	(Dynamic) Decentralized	Without RO	Adaptive key/message queries	Unbounded key/message tag repetitions	Adaptive user-corruption queries	Assumption + (any FE as building block)	Per-client CT size
[CDG ⁺ 18]	IP	✗	✓	✗	✓	✗	✗	SXDH	$O_\lambda(d)$
[ABG19]	IP	✗	✓	✓	✓	✓	✓	IPFE	$O_\lambda(d \cdot n)$
[LT19]	IP	✗	✓	✓	✓	✗	✗	LWE	$O_\lambda(d)$
[CDSG ⁺ 20]	IP	✗	✓	✗	✗	✓	✗	DDH + IPFE	$O_\lambda(d)$
[ABM ⁺ 20]	IP	✗	✓	✗	✓	✓	✗	DCR	$O_\lambda(d)$
[AGT21b]	IP	✓	✓	✗	✗	✗	✗	SXDH + FH-IPFE	$O_\lambda(d)$
[SV23]	IP	✓	✗	✓	✓	✓	✗	DLin	$O_\lambda(d)$
[NPS24]	IP	✓	✓	✗	✓	✗	✗	SXDH	$O_\lambda(d \cdot J)$
Our FH-IP DDFE	IP	✓	✓	✓	✓	✓	✗	SXDH + FH-IPFE	$O_\lambda(d + n)$
[ATY23]	AWS	✗	✓	✗	✗	✓	✗	MDDH + AWSw/IP-FE	$O_\lambda(N(kn_0 + n_1))$
Our AWS DDFE	AWS	✗	✓	✓	✗	✓	✗	SXDH + AWSw/IP-FE	$O_\lambda(N(kn_0 + n_1 + n))$

Fig. 1. Comparison with prior (Decentralized) MCFE schemes. The notation $O_\lambda(\cdot)$ indicates that terms related to the security parameter λ are hidden. We let d be an inner-product dimension, n be a $\text{poly}(\lambda)$ -large number of clients whose ciphertexts and decryption keys can be combined, N be a $\text{poly}(\lambda)$ -large number of AWS inputs, n_0 and n_1 be the input and output dimensions of ABPs, and k be the parameter of the MDDH assumption. The construction in [NPS24] set ups the number of repetitions of the same key tags in key queries to be a $\text{poly}(\lambda)$ -large value J . All schemes support incomplete key/message queries, either by design or through a generic upgrade.

1.3 Technical Overview

Challenges in Removing Trusted Setup. In the (decentralized) multi-client setting, one of the fundamental security requirements is that only inputs encrypted under a same message label and only keys generated under a same key label can be decrypted together, otherwise the decryption should output nothing.

Thus, a common approach in constructing (decentralized) MCFE schemes, including those for (function-hiding) inner products and attribute-weighted sums, is to rely on a correlated one-time-pad technique: the one-time pads should be freshly generated by each pair of message and key labels, as well as by each client, to independently randomize the information corresponding to each client’s private pair of input and key object.

Without obscuring the reliance on random oracles for its security, we can simplify the DDFE scheme for FH-IP in [AGT21b] as follows: leveraging a private use of a function-hiding IPFE scheme with IPE.Enc as the encryption algorithm, IPE.DKGen as the functional-key generation algorithm, and sk as the secret key; each client $\text{pk} \in \mathcal{U}$ encrypts a private vector $\mathbf{x}_{\text{pk}}$ as

$$\text{ct}_{\text{pk}} = \text{IPE.Enc}(\text{sk}_{\text{pk}}, [\mathbf{x}_{\text{pk}}, \mathbf{0}^d, h_{\ell_M, \mathcal{U}}, 0]_1)^2$$

where d denotes the inner-product dimension, $\mathbf{0}^d$ is a zero vector of dimension d reserved solely for the security proof, and $[h_{\ell_M, \mathcal{U}}]_1 = \mathcal{H}(\ell_M, \mathcal{U})$ where $\mathcal{H}$ is a hash function onto the group modeled as a random oracle. The decryption key for a private vector $\mathbf{y}_{\text{pk}}$ is generated as

$$\text{dk}_{\text{pk}} = \text{IPE.DKGen}(\text{sk}_{\text{pk}}, [\mathbf{y}_{\text{pk}}, \mathbf{0}^d, z_{\text{pk}, \ell_K}, 0]_2)$$

where z_{pk, ℓ_K} is generated on label ℓ_K by a pseudorandom zero-sharing (PZS) scheme between parties in $\mathcal{U}$ so that $\sum_{\text{pk} \in \mathcal{U}} z_{\text{pk}, \ell_K} = 0$. On one hand, the correctness holds as $\sum_{\text{pk} \in \mathcal{U}} z_{\text{pk}, \ell_K} \cdot h_{\ell_M, \mathcal{U}} = 0$ in the sum of all FH-IPFE decryptions.

On the other hand, via zero slots, and by using hybrids relying on the function-hiding security of each client’s FH-IPFE in the non-corrupted set $\mathcal{H}$ and each key label ℓ_K , one can move the term

² For the notation of elements in groups, we refer the reader to Section A.

$\llbracket z_{\text{pk}, \ell_K} \rrbracket_2$ from dk_{pk} to the term $\llbracket z_{\text{pk}, \ell_K} \cdot h_{\ell_M, \mathcal{U}} \rrbracket_1$ in ct_{pk} . At this step, one will have

$$\llbracket z_{\text{pk}, \ell_K} \cdot h_{\ell_M, \mathcal{U}} \rrbracket_1 \stackrel{\text{PZS, SXDH}}{\approx} \llbracket R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K} \rrbracket_1$$

where $R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K}$ are random subjected to $\sum_{\text{pk} \in \mathcal{H}} R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K} = -\sum_{\text{pk} \in \mathcal{U} \setminus \mathcal{H}} z_{\text{pk}, \ell_K} \cdot h_{\ell_M, \mathcal{U}}$. Each $R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K}$ then serves as a correlated one-time pad that randomizes each term $\mathbf{x}_{\text{pk}}^b \cdot \mathbf{y}_{\text{pk}}^b$ in the same way it randomizes $\mathbf{x}_{\text{pk}}^0 \cdot \mathbf{y}_{\text{pk}}^0$, where b is the challenge bit of the security game. It is important to note that for the SXDH assumption to hold, $\llbracket h_{\ell_M, \mathcal{U}} \rrbracket_1$ must be a uniformly random group element to all clients, which requires the use of a random oracle.

A more recent construction in [SV23] addresses this problem in the multi-client setting, in which a trusted key authority is necessitated. Extending this construction to the decentralized setting is challenging, as the assumption of a trusted key authority is, in fact, stronger than the assumption of a random oracle.

It is also worth noticing that the generic transformation from single-input FE to decentralized MCFE for function-revealing inner products in [ABG19] cannot be extended to the function-hiding setting, as the transformation necessitates each client to know the entire joint inner-product function. Furthermore, the technique used to avoid the random oracle in LWE-based MCFE in [LT19] is inapplicable due to the impossibility of lattice-based FE for function-hiding inner products [Üna20].

Overall, as zero-sharing is treated as *black-box* in all existing FE constructions [ABG19, CDSG⁺20, AGT21b, SV23], updating shares into fresh ones may require a common reference string to simultaneously maintain both correlation and corruption-resistance. Therefore, to achieve security in the standard model, we provide a novel framework for zero-sharing construction.

Updatable Pseudorandom Zero-Sharing. In the construction from [KDK11], each zero share for a user i at a label ℓ is $\text{share}_{i, \ell} = \sum_{j \in [n] \setminus i} (-1)^{i < j} \text{PRF}_{k_{i,j}}(\ell)$. We parse each term $(-1)^{i < j} \text{PRF}_{k_{i,j}}(\ell)$ that is generated with a pairwise PRF key $k_{i,j} = k_{j,i}$ as a pairwise seed. Then our approach can be described at a high level as follows: instead of randomizing each share using a random oracle, we randomize each of its pairwise seeds using another PRF, denoted PRF' , which is computed using the same key of the corresponding pair.

More formally, we provide a definition and security models for the new notion Updatable Pseudorandom Zero-Sharing (UZS) in Section 3. This formalization will emphasize the properties needed in constructing DDFE schemes.

We provide a brief overview of our DDH-based construction in Figure 2, which eventually leads to the security in the standard model for pairing-based DDFE constructions. A natural extension to the MDDH assumption, which could be applied, for example in AWS-DDFE [ATY23] could be achieved using a similar construction strategy. When compared to the pseudorandom zero-sharing schemes in earlier works [KDK11, BIK⁺17, ABG19], it is worth noticing that zero shares in our scheme are group elements that sum up to the group identity, instead of scalars.

Algorithm	Return
Setup(λ)	a group $\mathbb{G}$ and pseudorandom functions (PRF, PRF').
KeyGen($\mathbb{G}$, PRF, PRF')	$\text{sk}_i = (k_{i,j})_{j \in [n] \setminus i}$ are exchanged keys with j for each user i .
SeedGen(sk_i, ℓ_s)	$\text{seed}_{i, \ell_s} = (\llbracket (-1)^{i < j} \text{PRF}_{k_{i,j}}(\ell_s) \rrbracket)_{j \in [n] \setminus i}$.
TokGen(sk_i, ℓ_u)	$\text{token}_{i, \ell_u} = (\text{PRF}'_{k_{i,j}}(\ell_u))_{j \in [n] \setminus i}$.
SeedUpt($\text{seed}_{i, \ell_s}, \text{token}_{i, \ell_u}$)	$\text{seed}_{i, \ell_s \ell_u} = (\llbracket (-1)^{i < j} \text{PRF}_{k_{i,j}}(\ell_s) \cdot \text{PRF}'_{k_{i,j}}(\ell_u) \rrbracket)_{j \in [n] \setminus i}$.
ShareEval($\text{seed}_{i, \ell}$)	$\text{share}_{i, \ell} = \text{seed}_{i, \ell}^\top \cdot \mathbf{1}^{n-1}$.

Fig. 2. Construction for UZS in DDH groups between n parties

We show how the scheme achieves correctness and security using matrix representation:

- The matrix $\llbracket \mathbf{A}_{\ell_s} \rrbracket$ (see Figure 3) serves as a seeding matrix, with each i -th row representing seed_{i, ℓ_s} . This seeding matrix adopts an anti-symmetric form: $A_{\ell_s, (i, i)} = 0$ for all $i \in [n]$ and $A_{\ell_s, (i, j)} = -A_{\ell_s, (j, i)}$ for all $(i, j \in [n], i \neq j)$.

$$\llbracket \mathbf{A}_{\ell_s} \rrbracket = \begin{pmatrix} 0 & -\text{PRF}_{k_{1,2}}(\ell_s) & \cdots & -\text{PRF}_{k_{1,n}}(\ell_s) \\ \text{PRF}_{k_{2,1}}(\ell_s) & 0 & \cdots & -\text{PRF}_{k_{2,n}}(\ell_s) \\ \vdots & \vdots & \ddots & \vdots \\ \text{PRF}_{k_{n,1}}(\ell_s) & \text{PRF}_{k_{n,2}}(\ell_s) & \cdots & 0 \end{pmatrix} \begin{array}{l} \rightarrow \text{seed}_{1,\ell_s} = (\llbracket A_{\ell_s,(1,j)} \rrbracket)_{j \in [n] \setminus \{1\}} \\ \rightarrow \text{seed}_{2,\ell_s} = (\llbracket A_{\ell_s,(2,j)} \rrbracket)_{j \in [n] \setminus \{2\}} \\ \rightarrow \text{seed}_{n,\ell_s} = (\llbracket A_{\ell_s,(n,j)} \rrbracket)_{j \in [n] \setminus \{n\}} \end{array}$$

Fig. 3. Seeding matrix $\llbracket \mathbf{A}_{\ell_s} \rrbracket \in \mathbb{G}^{n \times n}$.

- The matrix $\mathbf{B}_{\ell_u}$ (see Figure 4) serves as an updating matrix, with each i -th row representing token_{i,ℓ_u} . This updating matrix adopts a symmetric form: $B_{\ell_u,(i,i)} = 0$ for all $i \in [n]$ and $B_{\ell_u,(i,j)} = B_{\ell_u,(j,i)}$ for all $(i,j \in [n], i \neq j)$.

$$\mathbf{B}_{\ell_u} = \begin{pmatrix} 0 & \text{PRF}'_{k_{1,2}}(\ell_u) & \cdots & \text{PRF}'_{k_{1,n}}(\ell_u) \\ \text{PRF}'_{k_{2,1}}(\ell_u) & 0 & \cdots & \text{PRF}'_{k_{2,n}}(\ell_u) \\ \vdots & \vdots & \ddots & \vdots \\ \text{PRF}'_{k_{n,1}}(\ell_u) & \text{PRF}'_{k_{n,2}}(\ell_u) & \cdots & 0 \end{pmatrix} \begin{array}{l} \rightarrow \text{token}_{1,\ell_u} = (B_{\ell_u,(1,j)})_{j \in [n]} \\ \rightarrow \text{token}_{2,\ell_u} = (B_{\ell_u,(2,j)})_{j \in [n]} \\ \rightarrow \text{token}_{n,\ell_u} = (B_{\ell_u,(n,j)})_{j \in [n]} \end{array}$$

Fig. 4. Updating matrix $\mathbf{B}_{\ell_s} \in \mathbb{Z}_p^{n \times n}$.

- The matrix $\llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket$ (see Figure 5) serves as an updated seeding matrix, with each i -th row representing $\text{seed}_{i,\ell_s || \ell_u}$. This updated matrix results from a Hadamard product between $\llbracket \mathbf{A}_{\ell_s} \rrbracket$ and $\mathbf{B}_{\ell_u}$, which preserves the anti-symmetric form from $\llbracket \mathbf{A}_{\ell_s} \rrbracket$.

$$\begin{array}{c} \llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket = \llbracket \mathbf{A}_{\ell_s} \odot \mathbf{B}_{\ell_u} \rrbracket \\ \text{PRF}_{\text{DDH}} \approx \end{array} \quad \begin{array}{c} \text{DDH holds for non-} \\ \text{corrupted pairs} \\ \{(i,j)\}_{i \neq 2, j \neq 2} \end{array}$$

$$\begin{pmatrix} 0 & -\llbracket \text{PRF}_{k_{1,2}}(\ell_s) \text{PRF}'_{k_{1,2}}(\ell_u) \rrbracket & \cdots & \llbracket -\text{RF}_{(1,n)}(\ell_s || \ell_u) \rrbracket \\ \llbracket \text{PRF}_{k_{2,1}}(\ell_s) \text{PRF}'_{k_{2,1}}(\ell_u) \rrbracket & 0 & \cdots & -\llbracket \text{PRF}_{k_{2,n}}(\ell_s) \text{PRF}'_{k_{2,n}}(\ell_u) \rrbracket \\ \vdots & \vdots & \ddots & \vdots \\ \llbracket \text{RF}_{(1,n)}(\ell_s || \ell_u) \rrbracket & \llbracket \text{PRF}_{k_{n,2}}(\ell_s) \text{PRF}'_{k_{n,2}}(\ell_u) \rrbracket & \cdots & 0 \end{pmatrix} \begin{array}{l} \rightarrow \text{PRF keys} \\ \{k_{2,i}\}_{i \in [n], i \neq 2} \\ \text{are corrupted} \end{array}$$

$$\downarrow$$

$$\text{PRF keys } \{k_{i,2}\}_{i \in [n], i \neq 2} \text{ are corrupted}$$

Fig. 5. Indistinguishability for entries in the updated seeding matrix $\llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket$, where RF denotes a random function. For simple illustration, we assume only user P_2 is corrupted by the adversary.

In both cases, when $\ell = \ell_s$ or $\ell = \ell_s || \ell_u$, each $\text{share}_{i,\ell}$ computes the sum of all entries on the i -th row of an *antisymmetric matrix*. Therefore, $\sum_{i \in [n]} \text{share}_{i,\ell}$ computes the sum of all entries in the matrix, resulting in the group identity $\llbracket 0 \rrbracket$. This implies the correctness of the scheme.

At a high level, the standard security of a pseudorandom zero-sharing scheme guarantees that when the adversary corrupts a subset of parties and computes their shares on its own, the shares of the remaining honest users are computationally indistinguishable from a correlated random distribution.

However, this security is insufficient for the security proofs in DDFE schemes. Our scheme not only satisfies this standard security but also provides indistinguishability from a random distribution for the updated shares: even when the adversary has access to the seeding matrix $\llbracket \mathbf{A}_{\ell_s} \rrbracket$ and thereby has access to all the shares of the honest users on ℓ_s , the indistinguishability still holds for the updated honest users' shares in $\llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket$.

The reason is that the product maintains the integrity of the honest entries from $\llbracket \mathbf{A}_{\ell_s} \rrbracket$ and $\mathcal{B}_{\ell_u}$ to $\llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket$. We then have $\llbracket \text{PRF}_{k_{i,j}}(\ell_s) \text{PRF}'_{k_{i,j}}(\ell_u) \rrbracket \stackrel{\text{PRF, DDH}}{\approx} \llbracket \text{RF}_{(i,j)}(\ell_s || \ell_u) \rrbracket$ for each honest pair (i, j) (see Figure 5). This implies that the honest shares in $\llbracket \mathbf{C}_{\ell_s || \ell_u} \rrbracket$ are independent from those in $\llbracket \mathbf{A}_{\ell_s} \rrbracket$. Moreover, by using the Multi-DDH assumption (see [CDG⁺18]), which tightly reduces to the DDH assumption using the random-self reducibility, the indistinguishability of the honest updated shares on $(\ell_s || \ell_u)$ holds for a polynomial number of labels ℓ_s given a label ℓ_u . Therefore, we obtain a security without RO that is applicable to DDFE schemes.

The cost of achieving DDFE schemes in the standard model will be an increase in ciphertext/functional key size per client by an additive factor of $O_\lambda(n)$. Each ciphertext now hides a row of the updating matrix, while each functional key hides a row of the seeding matrix. While this trade-off impacts the efficiency, it does not compromise the full-fledged functionality of our solution with respect to the open question on the feasibility of FH-IP-DDFE in the standard model, as posed in [SV23]. Moreover, it remains more efficient than the multiplicative overhead of $O_\lambda(n)$ in [ABG19] (see Figure 1). On the other hand, it is challenging to compress seeds and tokens, as the relation between updated shares does not hold anymore.

A concrete construction for the dynamic setting is provided in Section 3.3, which relies on a non-interactive key exchange protocol NIKE, a pseudorandom function PRF and the DDH assumption. Its security for a restricted setting of *one-time-update* and static corruption is provided in Theorem 1.

From UZS to DDFE Schemes Without RO. For each client $\text{pk} \in \mathcal{U}$, the encryption of $\mathbf{x}_{\text{pk}}$ under $(\mathcal{U}, \ell_M)$ and the generation of decryption key for $\mathbf{y}_{\text{pk}}$ under $(\mathcal{U}, \ell_K)$ can be briefly described as follows:

$$\begin{aligned} \text{ct}_{\text{pk}} &= \text{IPE.Enc}_{(1^{\rho(\lambda, d)} + |\mathcal{U}|, \text{sk}_{\text{pk}})}(\llbracket \mathbf{x}_{\text{pk}}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, \mathbf{0}^{\rho(\lambda, d)} \rrbracket_1); \\ \text{dk}_{\text{pk}} &= \text{IPE.DKGen}_{(1^{\rho(\lambda, d)} + |\mathcal{U}|, \text{sk}_{\text{pk}})}(\llbracket \mathbf{y}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, \mathbf{0}^{\rho(\lambda, d)} \rrbracket_2) \end{aligned}$$

Here, d denotes the inner-product dimension, $\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K} \rrbracket_2 = \text{seed}_{\text{pk}, \mathcal{U}}^{\ell_K}$ and $\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} = \text{token}_{\text{pk}, \mathcal{U}}^{\ell_M}$ are generated using the UZS scheme. The IPE scheme is instantiated for inner products of dimension $(\rho(\lambda, d) + |\mathcal{U}|)$ where $\rho(\lambda, d)$ is a polynomial in the security parameter λ and the inner-product dimension d . The zero-vector $\mathbf{0}^{\rho(\lambda, d)}$ is reserved exclusively for the security proof.

The correctness of the scheme holds since

$$\begin{aligned} \text{IPE.Dec}(\text{ct}_{\text{pk}}, \text{dk}_{\text{pk}}) &= \llbracket \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} + \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M \top} \cdot \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K} \rrbracket_T \\ &= \llbracket \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} \rrbracket_T + e(\llbracket 1 \rrbracket_1, \text{share}_{\text{pk}, \mathcal{U}}^{\ell_K || \ell_M}) \end{aligned}$$

by the bilinear-update property of the UZS scheme.

For the security proof, we use a sequence of hybrid games in Figure 9 and Figure 10, with the goal of gradually replacing $\mathbf{x}^b$ with $\mathbf{x}^0$ in every ciphertext under each message label ℓ_M . Eventually, we remove $\mathbf{y}^b$ and leave $\mathbf{y}^0$ in every honest functional key to fully eliminate the challenge bit b . The security of UZS applies through a formal reduction in $\mathbf{G}_{\ell_M, 3}^*$ (see Figure 10). It is important to emphasize that this strategy aims to achieve security for adaptive message and key queries, which differs from the strategies for selective ones used in previous works [AGT21b, SV23].

Using a similar approach with the UZS scheme, we also construct a selectively secure AWS-DDFE scheme in the standard model, as detailed in Section 5. However, since the technique for preserving adaptive security applies only to FH-IP-DDFE, we leave as an open question whether it is possible to achieve adaptively secure AWS-DDFE from any adaptively secure AWS/IP-FE scheme.

Adaptive Security Preserving. In [AGT21b, SV23], the constructions of FH-IP-MCFE that use FH-IPFE as a black-box achieve security for only selective message and key queries. For simplicity in illustrating the core ideas, we assume that under each label, there is at most one queried message and one queried key³. The primary obstacle to achieving adaptive security lies in a specific step within

³ A complete analysis considering multiple queries is provided in Section 4.

the hybrid games, where the challenger must encode both a zero share $R_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K}$ and the product of the challenge message and key $\mathbf{x}_{\text{pk},\ell_M}^b \cdot \mathbf{y}_{\text{pk},\ell_K}^b$ in the same slot. This forms a sum $\mathbf{x}_{\text{pk},\ell_M}^b \cdot \mathbf{y}_{\text{pk},\ell_K}^b + R_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K}$, which, due to the randomness of the zero share and the admissibility conditions, is indistinguishable from $\mathbf{x}_{\text{pk},\ell_M}^0 \cdot \mathbf{y}_{\text{pk},\ell_K}^0 + R_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K}$. From here, a sequence of symmetric games is applied to $\mathbf{x}_{\text{pk},\ell_M}^0$ and $\mathbf{y}_{\text{pk},\ell_K}^0$. Complexity leveraging is not possible in this case, as guessing key or message queries would result in exponential security loss, while the indistinguishability between the games before and after encoding that sum is guaranteed only by a computational assumption. To overcome this obstacle, the adaptively-secure constructions in [SV23, NPS24] are based on specific FH-IPFE schemes to leverage the underlying computational assumption. In another related work [Tom19], Tomida presented an MIFE construction for FH-IP that preserves adaptive security when derived from any FH-IPFE, but the extension of this MIFE construction to DDFE remains unknown.

Our approach to address this issue is that for each argument on ℓ_M , we introduce intermediate one-time pads $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}$ and $\mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M}$ to mask the message-containing slots in ciphertexts as $(\mathbf{x}_{\text{pk},\ell_M}^b + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M})$. The resulting offset, $-(\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M} \cdot \mathbf{y}_{\text{pk},\ell_K}^b + \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M} \cdot \mathbf{y}_{\text{pk},\ell_K}^0)$, is encoded and randomized using the zero share $R_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K}$ in the functional keys. We also introduce the Injection Lemma (see Lemma 2), which considers the randomness of both the one-time pads and the zero share, alongside the admissibility conditions, to enable a perfectly indistinguishable transition from $(\mathbf{x}_{\text{pk},\ell_M}^b + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M})$ to $(\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{x}_{\text{pk},\ell_M}^0 + \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M})$. From this point, a sequence of symmetric games is applied to $\mathbf{x}_{\text{pk},\ell_M}^0$ and $\mathbf{y}_{\text{pk},\ell_K}^0$. By using these intermediate one-time pads, we overcome the challenge of having to encode $\mathbf{x}_{\text{pk},\ell_M}^b$ and $\mathbf{y}_{\text{pk},\ell_K}^b$ in the same slot, thereby obtaining adaptive security for FH-IP-DDFE from any adaptively secure FH-IPFE.

2 Preliminaries

2.1 Notations

Given any $n \in \mathbb{N}$, we denote by $[n]$ the set of integers $\{1, \dots, n\}$. Given any set $\mathcal{A}$, $\mathcal{L}(\mathcal{A})$ will denote the set of finite lists of elements of $\mathcal{A}$, and $\mathcal{S}(\mathcal{A})$ will denote the set of finite subsets of $\mathcal{A}$. While both lists and sets are ordered by default, lists may contain repeated elements. We denote by $|\mathcal{A}|$ the cardinal of any finite set $\mathcal{A}$. For any vector $\mathbf{a} \in \mathbb{A}^n$, we denote by a_i the i -th component of $\mathbf{a}$. Similarly, for any matrix $\mathbf{A} \in \mathbb{A}^{m \times n}$, we denote by $A_{i,j}$ the component at the position (i, j) of $\mathbf{A}$.

We also use the following notation for condition-based function-selecting functions

$$[f_0/f_1]^{\text{con}}(\text{inp}) = \begin{cases} \text{out} \leftarrow f_0(\text{inp}) & \text{if } \text{con} = 0; \\ \text{out} \leftarrow f_1(\text{inp}) & \text{if } \text{con} = 1. \end{cases}$$

We defer other notations, group assumptions along with the definitions of arithmetic branching programs, pseudorandom functions, non-interactive key exchange, further remarks on DDFE (including all-or-nothing encapsulation) in Section A.

2.2 Dynamic Decentralized Functional Encryption

Definition 1 (Dynamic Decentralized Functional Encryption). A dynamic decentralized functional encryption scheme over a set of public keys $\mathcal{PK}$ for functionality $F: \mathcal{L}(\mathcal{PK} \times \mathcal{K}) \times \mathcal{L}(\mathcal{PK} \times \mathcal{M}) \rightarrow \{0, 1\}^*$ consists of five algorithms:

- **SetUp**(1^λ): On input a parameter 1^λ , it generates and outputs public parameters pp . Those parameters are implicit arguments to all the other algorithms.
- **KeyGen**($\cdot$): It generates and outputs a party's public key $\text{pk} \in \mathcal{PK}$ and the corresponding secret key sk_{pk} .
- **Enc**(sk_{pk}, m): On input a party's secret key sk_{pk} , a value $m \in \mathcal{M}$ to encrypt, it outputs a ciphertext $\text{ct}_{\text{pk},m}$.
- **DKGen**(sk_{pk}, k): On input a party's secret key sk_{pk} , a key space object $k \in \mathcal{K}$, it outputs a functional decryption key $\text{dk}_{\text{pk},k}$.
- **Dec**($(\text{dk}_{\text{pk},k_{\text{pk}}})_{\text{pk} \in \mathcal{U}_K}, (\text{ct}_{\text{pk},m_{\text{pk}}})_{\text{pk} \in \mathcal{U}_M}$): On input a finite list of functional decryption keys $(\text{dk}_{\text{pk},k_{\text{pk}}})_{\text{pk} \in \mathcal{U}_K}$, a finite list of ciphertexts $(\text{ct}_{\text{pk},m_{\text{pk}}})_{\text{pk} \in \mathcal{U}_M}$, where $\mathcal{U}_M, \mathcal{U}_K \in \mathcal{L}(\mathcal{PK})$ are the lists of senders and receivers respectively, it outputs a value $y \in \{0, 1\}^*$.

Correctness. For all parameters $\lambda \in \mathbb{N}$, all polynomial size lists $\mathcal{U}_M, \mathcal{U}_K \in \mathcal{L}(\mathcal{PK})$, $(\mathbf{pk}, k_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_K} \in \mathcal{L}(\mathcal{PK} \times \mathcal{K})$ and $(\mathbf{pk}, m_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_M} \in \mathcal{L}(\mathcal{PK} \times \mathcal{M})$, it holds that

$$\Pr [\text{Dec}((\mathbf{dk}_{\mathbf{pk}, k_{\mathbf{pk}}})_{\mathbf{pk} \in \mathcal{U}_K}, (\mathbf{ct}_{\mathbf{pk}, m_{\mathbf{pk}}})_{\mathbf{pk} \in \mathcal{U}_M}) = F((\mathbf{pk}, k_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_K}, (\mathbf{pk}, m_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_M})] = 1,$$

where the probability is taken over all algorithms.

In this work, we assume that each user is identified by a public key $\mathbf{pk}$, which it can generate on its own with the (unique) associated secret key, using KeyGen . Anyone can thus dynamically join the system, by publishing its public key.

Remark 1 (Empty lists). As in [CDSG⁺20], we denote by ϵ_K the empty list in $\mathcal{L}(\mathcal{PK} \times \mathcal{K})$, indicating that there is no key required. Furthermore, ϵ_M denotes the empty list in $\mathcal{L}(\mathcal{PK} \times \mathcal{M})$, indicating that there is no message required.

Definition 2 (Security for DDFE). For $\mathbf{xx} \in \{\text{sel-sta}, \text{sta}, \text{any}\}$, $\mathbf{yy} \in \{\text{sym}, \text{asym}\}$, a $\mathbf{xx}$ - $\mathbf{yy}$ -IND security game of DDFE for every PPT adversary $\mathcal{A}$ is defined with access to the oracles QNewHon , QEnc , QDKGen and QCor described below:

- Initialize: the challenger runs the setup algorithm $\mathbf{pp} \leftarrow \text{SetUp}(\lambda)$ and chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. It provides $\mathbf{pp}$ to the adversary $\mathcal{A}$;
- Participation creation queries $\text{QNewHon}()$: it generates $(\mathbf{pk}, \mathbf{sk}_{\mathbf{pk}}) \leftarrow \text{KeyGen}()$, stores the association $(\mathbf{pk}, \mathbf{sk}_{\mathbf{pk}})$ and returns $\mathbf{pk}$ to the adversary;
- Challenge message queries $\text{QEnc}(\mathbf{pk}, m^0, m^1)$: it outputs the ciphertext $\mathbf{ct}_m \leftarrow (\mathbf{sk}, m^b)$ where $\mathbf{sk}$ is associated with $\mathbf{pk}$. If $\mathbf{pk}$ is not associated with any secret key, nothing is returned;
- Challenge key queries $\text{QDKGen}(\mathbf{pk}, k^0, k^1)$: it outputs the decryption key $\mathbf{dk}_k \leftarrow \text{DKGen}(\mathbf{sk}, k^b)$ where $\mathbf{sk}$ is associated with $\mathbf{pk}$. If $\mathbf{pk}$ is not associated with any secret key, nothing is returned;
- Corruption queries $\text{QCor}(\mathbf{pk})$: it outputs the secret key $\mathbf{sk}$ associated to $\mathbf{pk}$. If $\mathbf{pk}$ is not associated with any secret key, nothing is returned;
- Finalize: $\mathcal{A}$ provides its guess b' on the bit b , and this procedure outputs the result β according to the admissibility condition given below.

Let $\mathcal{PK}$ be the set of parties on which $\text{QNewHon}()$ is queried, $\mathcal{C} \subset \mathcal{PK}$ be the set of corrupted parties, $\mathcal{H} = \mathcal{PK} \setminus \mathcal{C}$ be the set of honest (non-corrupted) participants at the end of the game. Finalize outputs the bit $\beta = (b' = b)$ if the Condition (*) is satisfied, otherwise Finalize outputs $\beta \xleftarrow{\$} \{0, 1\}$. Condition (*) holds if all the following conditions hold:

- there do not exist two lists of messages $(\mathbf{m}^0 = (\mathbf{pk}, m_{\mathbf{pk}}^0)_{\mathbf{pk} \in \mathcal{U}_M}, \mathbf{m}^1 = (\mathbf{pk}, m_{\mathbf{pk}}^1)_{\mathbf{pk} \in \mathcal{U}_M})$, including (ϵ_M, ϵ_M) , and two lists of keys $(\mathbf{k}^0 = (\mathbf{pk}, k_{\mathbf{pk}}^0)_{\mathbf{pk} \in \mathcal{U}_K}, \mathbf{k}^1 = (\mathbf{pk}, k_{\mathbf{pk}}^1)_{\mathbf{pk} \in \mathcal{U}_K})$, including (ϵ_K, ϵ_K) , such that
 - $F(\mathbf{k}^0, \mathbf{m}^0) \neq F(\mathbf{k}^1, \mathbf{m}^1)$;
 - $\forall \mathbf{pk} \in \mathcal{U}_M$, $[\text{QEnc}(\mathbf{pk}, m_{\mathbf{pk}}^0, m_{\mathbf{pk}}^1)$ was made and $\mathbf{pk} \in \mathcal{H}]$ or $[m_{\mathbf{pk}}^0 = m_{\mathbf{pk}}^1 \in \mathcal{M}$ and $\mathbf{pk} \in \mathcal{C}]$;
 - $\forall \mathbf{pk} \in \mathcal{U}_K$, $[\text{QDKGen}(\mathbf{pk}, k_{\mathbf{pk}}^0, k_{\mathbf{pk}}^1)$ was made and $\mathbf{pk} \in \mathcal{H}]$ or $[k_{\mathbf{pk}}^0 = k_{\mathbf{pk}}^1 \in \mathcal{K}$ and $\mathbf{pk} \in \mathcal{C}]$.
- when $\mathbf{xx} = \text{sel-sta}$: the adversary sends all its $\text{QNewHon}()$ queries in one shot. After that it sends in one shot all $\text{QEnc}(\mathbf{pk}, m^0, m^1)$, $\text{QDKGen}(\mathbf{pk}, k^0, k^1)$ and $\text{QCor}(\mathbf{pk})$ queries⁴;
- when $\mathbf{xx} = \text{sta}$: the adversary sends all its $\text{QNewHon}()$ queries in one shot. After that it sends in one shot all $\text{QCor}(\mathbf{pk})$ queries only;
- when $\mathbf{yy} = \text{sym}$: for $\mathbf{pk} \in \mathcal{C}$, the $\text{QDKGen}(\mathbf{pk}, k_{\mathbf{pk}}^0, k_{\mathbf{pk}}^1)$ and $\text{QEnc}(\mathbf{pk}, m_{\mathbf{pk}}^0, m_{\mathbf{pk}}^1)$ queries must satisfy $k_{\mathbf{pk}}^0 = k_{\mathbf{pk}}^1$ and $m_{\mathbf{pk}}^0 = m_{\mathbf{pk}}^1$, respectively.⁵

We say DDFE is $\mathbf{xx}$ - $\mathbf{yy}$ -IND-secure if given any parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, the following holds

$$\text{Adv}_{\text{DDFE}}^{\mathbf{xx}, \mathbf{yy}}(\mathcal{A}) := \left| \Pr[\beta = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

We also have $\text{sta-sym-IND-security}$ implies $\text{sel-sta-sym-IND-security}$.

⁴ This setting is equivalent to the selective setting defined in [CDSG⁺20].

⁵ The symmetric setting is a natural restriction for FH-IP-DDFE, as knowing $\mathbf{sk}_{\mathbf{pk}}$ from corruption queries allows any adversary to generate itself valid ciphertexts (or decryption keys) to learn the challenge decryption keys (or ciphertexts) from inner-products.

Definition 3 (Function-Hiding Inner-Product DDFE). A function-hiding IP-DDFE scheme over a pairing group $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p)$ is defined for a dimension $d \in \mathbb{N}$, a message bound $X < \frac{p}{2}$, a function bound $Y < \frac{p}{2}$, and label spaces $(\mathcal{L}_M, \mathcal{L}_K)$ as follows:

$$\mathcal{K} = \left(\mathbb{G}_2^{[-Y, Y]} \right)^d \times \mathcal{S}(\mathcal{PK}) \times \mathcal{L}_K; \quad \mathcal{M} = \left(\mathbb{G}_1^{[-X, X]} \right)^d \times \mathcal{S}(\mathcal{PK}) \times \mathcal{L}_M.$$

Parsing $m_{\text{pk}} = (\llbracket \mathbf{x}_{\text{pk}} \rrbracket_1, \mathcal{U}_{\text{pk}}, \ell_{\text{pk}})$ and $k_{\text{pk}} = (\llbracket \mathbf{y}_{\text{pk}} \rrbracket_2, \mathcal{U}_{\text{pk}}, \ell_{\text{pk}})$; then, one has

$$\begin{aligned} F(\epsilon_K, (\text{pk}, m_{\text{pk}})_{\text{pk} \in \mathcal{U}}) &= (\mathcal{U}_{\text{pk}}, \ell_{\text{pk}})_{\text{pk} \in \mathcal{U}}; \\ F((\text{pk}, k_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \epsilon_M) &= (\mathcal{U}_{\text{pk}}, \ell_{\text{pk}})_{\text{pk} \in \mathcal{U}} \end{aligned}$$

for any $\mathcal{U} \in \mathcal{L}(\mathcal{PK})$ and

$$F((\text{pk}, k_{\text{pk}})_{\text{pk} \in \mathcal{U}_K}, (\text{pk}, m_{\text{pk}})_{\text{pk} \in \mathcal{U}_M}) = \begin{cases} \sum_{\text{pk} \in \mathcal{U}_K} \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} & \text{if condition } (*) \\ \perp & \text{otherwise.} \end{cases}$$

FH-IP-DDFE condition $(*)$ is:

- $\mathcal{U}_K = \mathcal{U}_M = \mathcal{U}$;
- $\exists \ell_K \in \mathcal{L}_K, \forall \text{pk} \in \mathcal{U}, k_{\text{pk}} = (\llbracket \mathbf{y}_{\text{pk}} \rrbracket_2, \mathcal{U}, \ell_K)$;
- $\exists \ell_M \in \mathcal{L}_M, \forall \text{pk} \in \mathcal{U}, m_{\text{pk}} = (\llbracket \mathbf{x}_{\text{pk}} \rrbracket_1, \mathcal{U}, \ell_M)$.

Remark 2 (Single-Input Inner-Product). We denote by IPE a single-input FE for function-hiding inner products, that was defined as in Definition 3 for the case $|\mathcal{PK}| = 1$.

Definition 4 (Attribute-Weighted-Sum DDFE). A function-revealing AWS-DDFE scheme over a pairing group $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p)$ is defined for the class of arithmetic branching programs $\mathcal{F}_{n_0, n_1}^{\text{ABP}}$ and a label space $\mathcal{L}_M$ as follows:

$$\begin{aligned} \mathcal{K} &= \{ \mathbf{f} := (\text{pk}', f_{\text{pk}'})_{\text{pk}' \in \mathcal{U}_K} \text{ where } f_{\text{pk}'} \in \mathcal{F}_{n_0, n_1}^{\text{ABP}} \text{ and } \mathcal{U}_K \in \mathcal{S}(\mathcal{PK}) \} \\ \mathcal{M} &= \bigcup_{i \in \mathbb{N}} (\mathbb{Z}_p^{n_0} \times \mathbb{Z}_p^{n_1})_i \times \mathcal{S}(\mathcal{PK}) \times \mathcal{L}_M. \end{aligned}$$

Parsing $m_{\text{pk}} = ((\mathbf{x}_{\text{pk}, j}, \mathbf{z}_{\text{pk}, j})_{j \in [N_{\text{pk}}]}, \mathcal{U}_{\text{pk}}, \ell_{\text{pk}})$ and $k_{\text{pk}} = (\mathbf{f}_{\text{pk}}, \mathcal{U}_{\text{pk}})$; then, one has

$$\begin{aligned} F(\epsilon_K, (\text{pk}, m_{\text{pk}})_{\text{pk} \in \mathcal{U}}) &= ((\mathbf{x}_{\text{pk}, j})_{j \in [N_{\text{pk}}]}, \mathcal{U}_{\text{pk}}, \ell_{\text{pk}})_{\text{pk} \in \mathcal{U}}; \\ F((\text{pk}, k_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \epsilon_M) &= (\mathbf{f}_{\text{pk}}, \mathcal{U}_{\text{pk}})_{\text{pk} \in \mathcal{U}} \end{aligned}$$

for any $\mathcal{U} \in \mathcal{L}(\mathcal{PK})$ and

$$F((\text{pk}, k_{\text{pk}})_{\text{pk} \in \mathcal{U}_K}, (\text{pk}, m_{\text{pk}})_{\text{pk} \in \mathcal{U}_M}) = \begin{cases} \llbracket \sum_{\text{pk}' \in \mathcal{U}_K} \sum_{j \in [N_{\text{pk}'}]} f_{\text{pk}'}(\mathbf{x}_{\text{pk}', j})^\top \mathbf{z}_{\text{pk}', j} \rrbracket_T & \text{if } (*) \\ \perp & \text{otherwise.} \end{cases}$$

AWS-DDFE condition $(*)$ is:

- $\mathcal{U}_K = \mathcal{U}_M = \mathcal{U}$;
- $\exists \mathbf{f} \in (\text{pk}', f_{\text{pk}'})_{\text{pk}' \in \mathcal{U}}, \forall \text{pk} \in \mathcal{U}, k_{\text{pk}} = (\mathbf{f}, \mathcal{U})$;
- $\exists \ell_M \in \mathcal{L}_M, \forall \text{pk}' \in \mathcal{U}, m_{\text{pk}'} = ((\mathbf{x}_{\text{pk}', j}, \mathbf{z}_{\text{pk}', j})_{j \in [N_{\text{pk}'}]}, \mathcal{U}, \ell_M)$.

Definition 5 (Single-Input FE for AWSw/IP). Consider the case $|\mathcal{PK}| = 1$ in Definition 1 for the single-input setting, an FE for attribute-weighted sums with function-hiding inner product scheme over $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p)$ is defined for the class of arithmetic branching programs $\mathcal{F}_{n_0, n_1}^{\text{ABP}}$ as follows:

$$\mathcal{K} = \mathcal{F}_{n_0, n_1}^{\text{ABP}} \times \mathbb{G}_2^d; \quad \mathcal{M} = \bigcup_{i \in \mathbb{N}} (\mathbb{Z}_p^{n_0} \times \mathbb{Z}_p^{n_1})_i \times \mathbb{G}_1^d.$$

Then, one has

$$F(\epsilon_K, ((\mathbf{x}_j, \mathbf{z}_j)_{j \in [N]}, \llbracket \mathbf{s} \rrbracket_1)) = ((\mathbf{x}_j)_{j \in [N]}); \quad F((f, \llbracket \mathbf{t} \rrbracket_2), \epsilon_M) = f;$$

and

$$F((f, \llbracket \mathbf{t} \rrbracket_2), ((\mathbf{x}_j, \mathbf{z}_j)_{j \in [N]}, \llbracket \mathbf{s} \rrbracket_1)) = \llbracket \sum_{j \in [N]} f(\mathbf{x}_j)^\top \mathbf{z}_j + \mathbf{s}^\top \mathbf{t} \rrbracket_T.$$

We note that a single-input FE construction for AWSw/IP in the standard model is provided in [ATY23].

3 Updatable Pseudorandom Zero Sharing

In this section, we provide a definition and a security model for Updatable Pseudorandom Zero-Sharing, which serves as the key building block for later pairing-based DDFE constructions. A scheme in DDH groups that supports the bilinear update property will also be provided.

3.1 Definition

Definition 6 (Updatable Pseudorandom Zero Sharing). *Given a set of users $\mathcal{PK}$, a seeding label space $\mathcal{L}_S$ and an updating label space $\mathcal{L}_U$, an updatable pseudorandom zero sharing scheme UZS over a group $(\mathbb{A}, +)$ consists of six algorithms:*

- **SetUp**(λ): *On input a security parameter λ , it outputs public parameters pp . Those parameters are implicit arguments to all the other algorithms.*
- **KeyGen**($\cdot$): *It outputs a party's public key $\text{pk} \in \mathcal{PK}$ and the corresponding secret key sk_{pk} .*
- **SeedGen**($\text{sk}_{\text{pk}}, \mathcal{U}, \ell_s$): *On input a secret key sk_{pk} , a user set $\mathcal{U} \in \mathcal{S}(\mathcal{PK})$, and a seeding label $\ell_s \in \mathcal{L}_S$, it outputs a seed $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s}$ if $\text{pk} \in \mathcal{U}$. Otherwise, it outputs $\perp$.*
- **TokGen**($\text{sk}_{\text{pk}}, \mathcal{U}, \ell_u$): *On input a secret key sk_{pk} , a user set $\mathcal{U} \in \mathcal{S}(\mathcal{PK})$, and an updating label $\ell_u \in \mathcal{L}_U$, it outputs a token $\text{token}_{\text{pk}, \mathcal{U}}^{\ell_u}$ if $\text{pk} \in \mathcal{U}$. Otherwise, it outputs $\perp$.*
- **SeedUpt**($\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s}, \text{token}_{\text{pk}, \mathcal{U}}^{\ell_u}$): *On input a seed $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s}$ and a token $\text{token}_{\text{pk}, \mathcal{U}}^{\ell_u}$, it outputs a seed $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u}$.*
- **ShareEval**($\text{seed}_{\text{pk}, \mathcal{U}}^{\ell}$): *On input a seed $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell}$ for $\ell \in \mathcal{L}_S \cup (\mathcal{L}_S \times \mathcal{L}_U)$, it outputs a share $\text{share}_{\text{pk}, \mathcal{U}}^{\ell}$.*

Correctness. *For any security parameter $\lambda \in \mathbb{N}$, any $\ell_s \in \mathcal{L}_S$, any $\ell_u \in \mathcal{L}_U$, any $\mathcal{U} \in \mathcal{S}(\mathcal{PK})$, then it holds that $\Pr \left[\sum_{\text{pk} \in \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^{\ell_s} = \sum_{\text{pk} \in \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} = 0_{\mathbb{A}} \right] = 1$ where the probability is taken over all algorithms.*

Definition 7 (Correlated Pseudorandomness for UZS). *For $\text{xx} \in \{\text{otu}, \text{any}\}$, $\text{yy} \in \{\text{sta}, \text{adt}\}$, a xx-yy-IND security game of UZS for every PPT adversary $\mathcal{A}$ is defined with access to the oracles QNewHon , QCor , QTokGen , QSeedGen , and QShare described below:*

- *Initialize: the challenger runs the setup algorithm $\text{pp} \leftarrow \text{SetUp}(\lambda)$ and chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. It initializes the sets $\mathcal{H}$ and $\mathcal{C}$ of honest participants and corrupted participants respectively to $\emptyset$, and provides pp to the adversary $\mathcal{A}$;*
- *Participant creation queries $\text{QNewHon}(\cdot)$: it generates $(\text{pk}, \text{sk}_{\text{pk}}) \leftarrow \text{KeyGen}(\cdot)$ to simulate a new participant, stores the association $(\text{pk}, \text{sk}_{\text{pk}})$ in the set $\mathcal{H}$, and returns pk to the adversary;*
- *Corruption queries $\text{QCor}(\text{pk})$: it moves the association $(\text{pk}, \text{sk}_{\text{pk}})$ from $\mathcal{H}$ to $\mathcal{C}$ and returns the secret key sk . If pk is not associated with any secret key in $\mathcal{H}$, then nothing is returned;*
- *Seed generation queries $\text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_s)$: if $[\text{pk} \notin \mathcal{U}]$ or $[\ell_s \notin \mathcal{L}_S]$ or $[\text{pk}$ is not associated with any secret key in either $\mathcal{H}$ or $\mathcal{C}]$, then nothing is returned. It returns $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s} \leftarrow \text{SeedGen}(\text{sk}, \mathcal{U}, \ell_s)$ to the adversary otherwise.*
- *Token generation queries $\text{QTokGen}(\text{pk}, \mathcal{U}, \ell_u)$: if $[\text{pk} \notin \mathcal{U}]$ or $[\ell_u \notin \mathcal{L}_U]$ or $[\text{pk}$ is not associated with any secret key in either $\mathcal{H}$ or $\mathcal{C}]$, nothing is returned. It returns $\text{token}_{\text{pk}, \mathcal{U}}^{\ell_u} \leftarrow \text{TokGen}(\text{sk}, \mathcal{U}, \ell_u)$ to the adversary otherwise.*
- *Challenge share queries $\text{QShare}(\mathcal{U}, \ell)$: if $[\ell \notin \mathcal{L}_S \cup (\mathcal{L}_S \times \mathcal{L}_U)]$ or $[|\mathcal{H} \cap \mathcal{U}| = 0]$, nothing is returned. We define the following share distributions:*

- *A pseudorandom share generation algorithm $\text{PShareGen}(\mathcal{S}, \mathcal{U}, \ell)$: it outputs $\perp$ if $\mathcal{S} \not\subseteq \mathcal{U}$, otherwise the challenger generates $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell}$ as follows:*

- * *if $\ell = \ell_s \in \mathcal{L}_S$: $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s} = \text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_s)$;*
- * *if $\ell = \ell_s || \ell_u \in \mathcal{L}_S \times \mathcal{L}_U$: $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell} = \text{SeedUpt}(\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s}, \text{token}_{\text{pk}, \mathcal{U}}^{\ell_u})$*

$$\text{where } \begin{cases} \text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s} = \text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_s) \\ \text{token}_{\text{pk}, \mathcal{U}}^{\ell_u} = \text{QTokGen}(\text{pk}, \mathcal{U}, \ell_u) \end{cases}$$

and outputs $\text{share}_{\text{pk}, \mathcal{U}}^{\ell} \leftarrow \text{ShareEval}(\text{seed}_{\text{pk}, \mathcal{U}}^{\ell})$ for all $\text{pk} \in \mathcal{S}$.

- A uniformly correlated random distribution for any $(\mathcal{S}, \mathcal{U}, \ell)$ where $\mathcal{S} \subset \mathcal{U}$

$$\mathcal{R}_{\mathcal{S}, \mathcal{U}}^\ell := \left\{ (s_{\text{pk}})_{\text{pk} \in \mathcal{S}} \stackrel{\$}{\leftarrow} \mathbb{A}^{|\mathcal{S}|} \mid \sum_{\text{pk} \in \mathcal{S}} s_{\text{pk}} = - \sum_{\text{pk} \in \mathcal{U} \setminus \mathcal{S}} t_{\text{pk}} \right\}$$

where $(t_{\text{pk}})_{\text{pk} \in \mathcal{U} \setminus \mathcal{S}} = \text{PShareGen}(\mathcal{U} \setminus \mathcal{S}, \mathcal{U}, \ell)$.

Then,

- if $b = 0$, the challenger returns $(\text{share}_{\text{pk}, \mathcal{U}}^\ell)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \leftarrow \text{PShareGen}(\mathcal{H} \cap \mathcal{U}, \mathcal{U}, \ell)$ to the adversary;
- if $b = 1$, the challenger returns $(\text{share}_{\text{pk}, \mathcal{U}}^\ell)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \leftarrow \mathcal{R}_{\mathcal{H} \cap \mathcal{U}, \mathcal{U}}^\ell$ to the adversary;
- Finalize: $\mathcal{A}$ provides its guess b' on the bit b , and this procedure outputs the result β of the game, according to the admissibility condition below which aims at preventing trivial wins.

Let $\mathcal{H}$ and $\mathcal{C}$ be the sets of honest users and corrupted users at the end of the game respectively. Finalize outputs the bit $\beta = (b' = b)$ if the Condition (*) is satisfied, otherwise Finalize outputs $\beta \stackrel{\$}{\leftarrow} \{0, 1\}$. Condition (*) holds if all the following conditions hold:

- there are no $\text{QCor}(\text{pk})$ queries after a $\text{QShare}(\mathcal{U}, \ell)$ query was made;
- there does not exist $\ell_s \in \mathcal{L}_S$ such that a $\text{QShare}(\mathcal{U}, \ell_s)$ query and a $\text{QSeedGen}(\text{pk}, \ell_s)$ query are sent;
- there does not exist $(\ell_s, \ell_u) \in \mathcal{L}_S \times \mathcal{L}_U$ such that a $\text{QShare}(\mathcal{U}, \ell_s || \ell_u)$ query and a $\text{QTokGen}(\text{pk}, \ell_u)$ query are sent;
- when $\text{xx} = \text{otu}$: for any $\mathcal{U}$, the queries of the form $\text{QShare}(\mathcal{U}, \ell_s || \cdot)$ can be sent for only one $\ell_u \in \mathcal{L}_U$.
- when $\text{yy} = \text{sta}$: the adversary sends all its $\text{QNewHon}()$ queries in one shot. After that it sends in one shot all $\text{QCor}(\text{pk})$ queries.

We say UZS is xx-yy-IND-secure if given any parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, the following holds

$$\text{Adv}_{\text{UZS}}^{\text{xx-yy}}(\mathcal{A}) = \left| \Pr[\beta = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

The security model described above captures the security of a standard pseudorandom zero-sharing scheme: the adversary has full access to the oracles and can win the game by only distinguishing the pseudorandom non-updated shares from the correlated random ones. Additionally, Condition (*) does not prevent the adversary from querying the seeds that result in the updated challenge shares. This intuitively means that even when the adversary has access to the seeds of honest users, the resulting updated shares remain indistinguishable from a correlated random distribution.

On the other hand, extending a standard pseudorandom zero-sharing scheme over $\mathcal{L} = \mathcal{L}_S \times \mathcal{L}_U$ to an updatable one can be achieved by using its share generation algorithm to create new shares on concatenated labels of the form $(\ell_s || \ell_u)$. However, this approach may require operations that are more complex than those allowed in pairing groups. Therefore, we specify a more pairing-friendly property for the updating algorithm of an UZS scheme.

Definition 8 (Bilinear Update). An updatable pseudorandom zero sharing scheme $\text{UZS} = (\text{SetUp}, \text{KeyGen}, \text{SeedGen}, \text{TokGen}, \text{SeedUpt}, \text{ShareEval})$ of security parameter λ is said to satisfy the bilinear update property if each seed is of the form $\llbracket \mathbf{a} \rrbracket \in \mathbb{A}^{\rho(\lambda)}$, each token is of the form $\mathbf{b} \in \mathbb{Z}^{\rho(\lambda)}$ where ρ is a λ -dependent parameter, and for any $\text{pk} \in \mathcal{PK}$, any $\ell_s \in \mathcal{L}_S$, and any $\ell_u \in \mathcal{L}_U$, it holds that

$$\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket \odot_\lambda \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u} = \text{SeedUpt}(\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u}),$$

where $\odot_\rho : \mathbb{A}^\rho \times \mathbb{Z}^\rho \rightarrow \mathbb{A}^\rho$ is an entry-wise bilinear map.

3.2 Construction in DDH Groups

Let $\text{PRF} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be a pseudorandom function, NIKE be a non-interactive key exchange protocol, $\mathcal{L}_S$ be a seeding label space, and $\mathcal{L}_U$ be an updating label space. A construction of UZS is described in Figure 6. We emphasize that zero shares in the scheme are group elements that sum up to the group identity.

Construction:

- **Setup**(1^λ): It generates $\mathcal{G} \leftarrow \text{GGen}(1^\lambda)$ and $\text{NIKE.pp} \leftarrow \text{NIKE.SetUp}(1^\lambda)$ and returns

$$\text{pp} = (\mathcal{G}, \text{NIKE.pp}, \text{PRF}, \mathcal{L}_S, \mathcal{L}_U).$$

The parameters pp are implicit to other algorithms.

- **KeyGen**(): Each user samples
 - NIKE keys $(\text{pk}, \text{NIKE.sk}_{\text{pk}}) \leftarrow \text{NIKE.KeyGen}()$;
 - a shared key $k_{\text{pk}, \text{pk}'} \leftarrow \text{NIKE.SharedKey}(\text{pk}', \text{NIKE.sk}_{\text{pk}})$ for each published $\text{pk}' \in \mathcal{PK} \setminus \{\text{pk}\}$.
 It returns

$$(\text{pk}, \text{sk}_{\text{pk}}) = (\text{pk}, (\text{NIKE.sk}_{\text{pk}}, (k_{\text{pk}, \text{pk}'}_{\text{pk}' \in \mathcal{PK} \setminus \{\text{pk}\}}))$$

- **SeedGen**($\text{sk}_{\text{pk}}, (\mathcal{U}, \ell_s)$): It computes

$$\mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} = \left((-1)^{\text{pk} < \text{pk}'} \text{PRF}_{k_{\text{pk}, \text{pk}'}}("s" || \mathcal{U} || \ell_s) \right)_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}}^{\textcolor{red}{a}},$$

and returns $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s} = \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket$. If $\text{pk} \notin \mathcal{U}$, it returns $\perp$.

- **TokGen**($\text{sk}_{\text{pk}}, (\mathcal{U}, \ell_u)$): It computes

$$\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u} = \left(\text{PRF}_{k_{\text{pk}, \text{pk}'}}("u" || \mathcal{U} || \ell_u) \right)_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}},$$

and returns $\text{token}_{\text{pk}, \mathcal{U}}^{\ell_u} = \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u}$. If $\text{pk} \notin \mathcal{U}$, it returns $\perp$.

- **SeedUpt**($\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s}, \text{token}_{\text{pk}, \mathcal{U}}^{\ell_u}$): It parses

- $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s} = \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket$,
- $\text{token}_{\text{pk}, \mathcal{U}}^{\ell_u} = \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u}$,
- $\rho = |\mathcal{U}| - 1$;

and returns

$$\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} = \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket \odot_\rho \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u}.$$

- **ShareEval**($\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u}$): It parses $\rho = |\mathcal{U}| - 1$, $\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} = \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} \rrbracket \in \mathbb{G}^\rho$, and returns

$$\text{share}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} = \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u} \rrbracket.$$

^a The PRF function can be instantiated separately using two distinct strings: "s" for seed generation and "u" for updating-token generation.

Fig. 6. Updatable Pseudorandom Zero-Sharing in DDH groups.

Correctness. Given $\lambda \in \mathbb{N}$, $\text{pp} \leftarrow \text{SetUp}(1^\lambda)$, $(\text{pk}, \text{sk}_{\text{pk}}) \leftarrow \text{KeyGen}()$ $\forall \text{pk} \in \mathcal{PK}$, $\mathcal{U} \in \mathcal{S}(\mathcal{PK})$, any labels $\ell_s \in \mathcal{L}_S$ and $\ell_u \in \mathcal{L}_U$, from the above scheme, one has

$$\begin{aligned} \sum_{\text{pk} \in \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^{\ell_s} &= \sum_{\text{pk} \in \mathcal{U}} \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket = \sum_{\text{pk} \in \mathcal{U}} \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket (-1)^{\text{pk} < \text{pk}'} c_{\text{pk}, \text{pk}'} \rrbracket \\ &= \sum_{\substack{(\text{pk}, \text{pk}') \in \mathcal{U}^2 \\ \text{pk} \neq \text{pk}'}} \llbracket (-1)^{\text{pk} < \text{pk}'} c_{\text{pk}, \text{pk}'} + (-1)^{\text{pk}' < \text{pk}} c_{\text{pk}', \text{pk}} \rrbracket \\ &= \sum_{\substack{(\text{pk}, \text{pk}') \in \mathcal{U}^2 \\ \text{pk} \neq \text{pk}'}} \llbracket 0 \rrbracket = \llbracket 0 \rrbracket. \end{aligned}$$

where $c_{\text{pk}, \text{pk}'} := \text{PRF}_{k_{\text{pk}, \text{pk}'}}("s" || \mathcal{U} || \ell_s)$ and $c_{\text{pk}', \text{pk}} = c_{\text{pk}, \text{pk}'}$. Similarly, one has

$$\begin{aligned} \sum_{\text{pk} \in \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u} &= \sum_{\text{pk} \in \mathcal{U}} \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \cdot b_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_u} \rrbracket \\ &= \sum_{\text{pk} \in \mathcal{U}} \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket (-1)^{\text{pk} < \text{pk}'} c'_{\text{pk}, \text{pk}'} \rrbracket \\ &= \sum_{\substack{(\text{pk}, \text{pk}') \in \mathcal{U}^2 \\ \text{pk} \neq \text{pk}'}} \llbracket 0 \rrbracket = \llbracket 0 \rrbracket. \end{aligned}$$

where $c'_{\text{pk}, \text{pk}'} := \text{PRF}_{k_{\text{pk}, \text{pk}'}}("s" || \mathcal{U} || \ell_s) \cdot \text{PRF}_{k_{\text{pk}, \text{pk}'}}("u" || \mathcal{U} || \ell_u)$ and $c'_{\text{pk}', \text{pk}} = c'_{\text{pk}, \text{pk}'}$. $\square$

Remark 3 (Bilinear Update). The UZS scheme in the above construction satisfies the bilinear update property in Definition 8.

3.3 Security Analysis

Theorem 1 (Indistinguishability for UZS). *If NIKE is a IND-secure non-interactive key exchange protocol, and the DDH assumption holds in $\mathbb{G}$, then the UZS scheme constructed in Section 3.2 is otu-sta-IND secure (as in Definition 7) in the standard model.*

Proof. In the static corruption game, we can fix $\mathcal{PK}$ to be the set of parties generated by $\text{QNewHon}()$, $\mathcal{C}$ to be the set of corrupted parties and $\mathcal{H} = \mathcal{PK} \setminus \mathcal{C}$ to be the set of honest parties. Let q_{QNewHon} and q_{QShare} be the number of QNewHon and QShare queries respectively. We consider two cases separately: $|\mathcal{H}| < 2$ and $|\mathcal{H}| \geq 2$.

The case of $|\mathcal{H}| < 2$. There is no information about b in this case: the output of $\text{QShare}(\mathcal{U}, \ell)$ does not return $\perp$ only if $|\mathcal{H} \cap \mathcal{U}| = 1$, then the distribution $\mathcal{R}_{S, \mathcal{U}}^\ell$ contains a single value to assign to the honest share. This value is equal to $\text{PShareGen}(\mathcal{H} \cap \mathcal{U}, \mathcal{U}, \ell)$.

The case of $|\mathcal{H}| \geq 2$. We proceed via a hybrid argument by using the games described in Figure 12. In this argument, the game $\mathbf{G}_0$ corresponds to the otu-sta-IND security game as defined in Definition 7, and the game $\mathbf{G}_3$ corresponds to the case where the adversary's advantage is 0 since there is no challenge bit b . Given $\lambda \in \mathbb{N}$, we denote by Adv_i the advantage of a PPT adversary $\mathcal{A}$ running in time t in each game $\mathbf{G}_i$, and Adv_{xx} be the best advantage of any PPT adversary running in time t against the primitive xx that is setup with λ .

Game $\mathbf{G}_1$: The change is that for each $(\text{pk}, \text{pk}') \in \mathcal{H}^2$, the challenger uses uniformly random shared keys $k_{\text{pk}, \text{pk}'}$ in answering to $\text{QSeedGen}(\text{pk}^*, \mathcal{U}, \ell_u)$, $\text{QTokGen}(\text{pk}^*, \mathcal{U}, \ell_s)$ queries for $\text{pk}^* \in \{\text{pk}, \text{pk}'\}$ (then in $\text{QShare}(\mathcal{U}, \ell)$ queries). The indistinguishability is implied by the security of the non-interactive key exchange protocol, given in Lemma 5.

Game $\mathbf{G}_2$: The change is that for each $(\text{pk}, \text{pk}') \in \mathcal{H}^2$, the challenger uses a random function $\text{RF}_{\text{pk}, \text{pk}'} = \text{RF}_{\text{pk}', \text{pk}}$ instead of $\text{PRF}_{k_{\text{pk}, \text{pk}'}}$ in answering $\text{QSeedGen}(\text{pk}^*, \mathcal{U}, \ell_u)$, $\text{QTokGen}(\text{pk}^*, \mathcal{U}, \ell_s)$ for $\text{pk}^* \in \{\text{pk}, \text{pk}'\}$, (then in $\text{QShare}(\mathcal{U}, \ell)$ queries). The indistinguishability is implied by the security of the pseudorandom functions, given in Lemma 6.

Game $\mathbf{G}_3$: The change is that for each $\text{QShare}(\mathcal{U}, \ell)$ query, the challenger answers independently from b by sampling $(\text{share}_{\text{pk}, \ell})_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \xleftarrow{\$} \mathcal{R}_{\mathcal{H} \cap \mathcal{U}, \mathcal{U}}^\ell$, where the distribution $\mathcal{R}_{\mathcal{H} \cap \mathcal{U}, \mathcal{U}}^\ell$ is defined in the QShare oracle in Definition 7. The indistinguishability is implied by the DDH assumption, given in Lemma 1.

From the transitions above, one completes the theorem. $\square$

We show the main proof technique that lies in the following lemma.

Lemma 1 (UZS: Transition from $\mathbf{G}_2$ to $\mathbf{G}_3$). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is*

$$|\text{Adv}_2 - \text{Adv}_3| \leq \frac{1}{2} q_{\text{QNewHon}}(q_{\text{QNewHon}} - 1) q_{\text{QShare}} \cdot \text{Adv}_{\text{DDH}}(t + 4q_{\text{QShare}} \times t_{\mathbb{G}}).$$

Proof. For every $\mathcal{U}$ queried in the form of $\text{QShare}(\mathcal{U}, \cdot)$ such that $|\mathcal{H} \cap \mathcal{U}| \geq 2$, we use multiple hybrid games that go all over the pairs in $\mathcal{P}_{\mathcal{U}} = \{(\text{pk}, \text{pk}') := (\text{pk}', \text{pk}) \in (\mathcal{H} \cap \mathcal{U})^2, \text{pk} \neq \text{pk}'\}$. We assume that all pairs in $\mathcal{P}_{\mathcal{U}}$ are bijectively mapped by κ to $[q_{\mathcal{U}}]$ for some integer $q_{\mathcal{U}} \geq 1$. For $i \in [q_{\mathcal{U}}]$, we define the following sequence of games as follows:

Game $\mathbf{G}_{2, \mathcal{U}, i}$: In this game, for any $\text{QShare}(\mathcal{U}, \ell_s || \ell_u^*)$ query⁶, when $b = 0$, for all $\text{pk} \in \mathcal{H} \cap \mathcal{U}$, the challenger computes

$$\begin{aligned} \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s} \rrbracket &= \text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_s), \quad \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_u^*} = \text{QTokGen}(\text{pk}, \mathcal{U}, \ell_u^*), \\ \text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u^*} &= \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u^*} \rrbracket \text{ where} \\ &\begin{cases} a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \xleftarrow{\$} \mathbb{Z}_p & \text{if } (\text{pk}, \text{pk}') \in \mathcal{P}_{\mathcal{U}} \text{ and } \kappa(\text{pk}, \text{pk}') < i \\ a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \xleftarrow{\$} \mathbb{Z}_p & \text{if } (\text{pk}, \text{pk}') \in \mathcal{P}_{\mathcal{U}} \text{ and } \kappa(\text{pk}, \text{pk}') = i \\ a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} = a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \cdot b_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_u^*} & \text{if } (\text{pk}, \text{pk}') \in \mathcal{P}_{\mathcal{U}} \text{ and } \kappa(\text{pk}, \text{pk}') > i \\ a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} = a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \cdot b_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_u^*} & \text{if } (\text{pk}, \text{pk}') \notin \mathcal{P}_{\mathcal{U}} \end{cases} \end{aligned}$$

and outputs $\text{share}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u^*} = \text{ShareEval}(\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_s || \ell_u^*})$. The change between $\mathbf{G}_{2, i-1}$ and $\mathbf{G}_{2, i}$ is in the PShareGen algorithm (defined by the QShare oracle in Definition 7) and is highlighted in gray.

For every $\mathcal{U}$, we assume that $\mathbf{G}_{2, \mathcal{U}, 0}$ is the game where $\text{QShare}(\mathcal{U}, \cdot)$ is the same as in $\mathbf{G}_2$. For each transition from $\mathbf{G}_{2, \mathcal{U}, i-1}$ to $\mathbf{G}_{2, \mathcal{U}, i}$ for $i \in [q_{\mathcal{U}}]$, we build an adversary $\mathcal{B}$ against the multi-DDH assumption (see Definition 10), which can be described as follows:

- To answer $\text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_s)$ and $\text{QSeedGen}(\text{pk}', \mathcal{U}, \ell_s)$ queries, the adversary $\mathcal{B}$ implicitly uses $\llbracket c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket := Y_{\ell_s} \xleftarrow{\$} \mathbb{G}$ from the multi-DDH when $\kappa(\text{pk}, \text{pk}') = i$ to compute $\llbracket a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket = \llbracket (-1)^{\text{pk} < \text{pk}'} c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket$ and $\llbracket a_{\text{pk}', \mathcal{U}, \text{pk}}^{\ell_s} \rrbracket = \llbracket (-1)^{\text{pk}' < \text{pk}} c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket$ respectively.
- To answer $\text{QShare}(\mathcal{U}, \ell_s || \ell_u^*)$ queries, if $b = 0$, $\mathcal{B}$ implicitly uses $\llbracket c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket := Z_{\ell_s}$ from the multi-DDH instance when $\kappa(\text{pk}, \text{pk}') = i$ to compute $\llbracket a_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket = \llbracket (-1)^{\text{pk} < \text{pk}'} c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket$ and $\llbracket a_{\text{pk}', \mathcal{U}, \text{pk}}^{\ell_s || \ell_u^*} \rrbracket = \llbracket (-1)^{\text{pk}' < \text{pk}} c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket$ respectively.
- $\mathcal{B}$ outputs $\mathcal{A}$'s guess for the challenge bit $\text{DDH}^{\text{multi}}.b$.

The adversary $\mathcal{B}$ has a complete multi-DDH challenge $\mathcal{D} = (X, (Y_{\ell_s})_{\ell_s}, (Z_{\ell_s})_{\ell_s})$ where X can be implicitly considered as $\llbracket b_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_u^*} \rrbracket \xleftarrow{\$} \mathbb{G}$.

- When $\text{DDH}^{\text{multi}}.b = 0$, one has $\llbracket c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket = Z_{\ell_s} = \text{CDH}(X, Y_{\ell_s}) = \llbracket c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s} \rrbracket \cdot b_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_u^*}$, which corresponds to $\mathbf{G}_{2, \mathcal{U}, i-1}$.
- When $\text{DDH}^{\text{multi}}.b = 1$, one has $\llbracket c_{\text{pk}, \mathcal{U}, \text{pk}'}^{\ell_s || \ell_u^*} \rrbracket = Z_{\ell_s} \xleftarrow{\$} \mathbb{G}$, which corresponds to $\mathbf{G}_{2, \mathcal{U}, i}$.

⁶ In the one-time-update setting, for each $\mathcal{U}$, ℓ_s can change while ℓ_u^* is fixed.

Therefore, the computational gap between each $\mathbf{G}_{2.\mathcal{U}.i-1}$ and $\mathbf{G}_{2.\mathcal{U}.i}$ happens only when $b = 0$ and is then bounded by $\frac{1}{2} \cdot \text{Adv}_{\text{DDH}}(\lambda, t + 4q_{\text{QShare}} \cdot t_{\mathbb{G}})$.

The last step is to show that for the last $\mathcal{U}^*$ queried to $\text{QShare}(\cdot, \cdot)$, one has $\mathbf{G}_{2.\mathcal{U}^*.q_{\mathcal{U}^*}} = \mathbf{G}_3$. It suffices to describe the case $b = 0$. We note that $\text{QShare}(\mathcal{U}, \ell) = (\text{share}_{\text{pk}, \mathcal{U}}^\ell)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ where $\text{share}_{\text{pk}, \mathcal{U}}^\ell = \sum_{\text{pk}' \in \mathcal{U} \setminus \{\text{pk}\}} \llbracket a_{\text{pk}, \mathcal{U}, \text{pk}'}^\ell \rrbracket$. By all transitions until $\mathbf{G}_{2.\mathcal{U}^*.q_{\mathcal{U}^*}}$, we have $a_{\text{pk}, \mathcal{U}, \text{pk}'}^\ell \xleftarrow{\$} \mathbb{Z}_p$ for any pair of honest users (pk, pk') , any set $\mathcal{U}$, any label ℓ . As $a_{\text{pk}', \mathcal{U}, \text{pk}}^\ell = -a_{\text{pk}, \mathcal{U}, \text{pk}'}^\ell$, then

$$\left(\text{share}_{\text{pk}, \mathcal{H} \cap \mathcal{U}}^{0, \ell} := \sum_{\text{pk}' \in \mathcal{H} \cap \mathcal{U} \setminus \{\text{pk}\}} a_{\text{pk}, \mathcal{U}, \text{pk}'}^\ell \right)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$$

are uniformly random shares of zero among users in $\mathcal{H} \cap \mathcal{U}$. Therefore, one has

$$\left(\text{share}_{\text{pk}, \mathcal{U}}^\ell := \sum_{\text{pk}' \in \mathcal{C} \cap \mathcal{U}} a_{\text{pk}, \mathcal{U}, \text{pk}'}^\ell + \text{share}_{\text{pk}, \mathcal{H} \cap \mathcal{U}}^{0, \ell} \right)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$$

are uniformly random shares of $-\sum_{\text{pk}' \in \mathcal{C} \cap \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^\ell$, which is identical to the distribution $\mathcal{R}_{\mathcal{H} \cap \mathcal{U}}^\ell$. Since the number of sets $\mathcal{U}$ queried to $\text{QShare}(\cdot, \ell)$ for $\ell \in \mathcal{L}_S \times \mathcal{L}_U$ is bounded by q_{QShare} , and each $q_{\mathcal{U}}$ is bounded by $(q_{\text{QNewHon}} - q_{\text{QCor}})$, one obtains $|\text{Adv}_2 - \text{Adv}_3| \leq \frac{1}{2} (q_{\text{QNewHon}} - q_{\text{QCor}}) q_{\text{QShare}} \cdot \text{Adv}_{\text{DDH}}(\lambda, t + 4q_{\text{QShare}} \times t_{\mathbb{G}})$. $\square$

4 Function-Hiding Inner-Product DDFE

In this section, we construct a DDFE scheme for function-hiding inner products. As an additional independent contribution, we present a lemma that supports transforming an IPE scheme into a DDFE scheme in a black-box manner while preserving the adaptive security of the original scheme. By simultaneously applying the UZS scheme and this lemma to construction, we achieve an FH-IP-DDFE scheme that is adaptively secure in the standard model.

4.1 Lemma for Adaptive Security

We first state a remark from the admissibility for FH-IP-DDFE, which will be extremely useful for proofs of lemmas and theorems in our DDFE construction.

Remark 4 (Invariance in FH-IP-DDFE). From DDFE security model (Definition 2) and FH-IP functionality (Definition 3), given any admissible adversary against FH-IP-DDFE and any challenge bit b , Condition (*) implies that among encryption queries under $(\ell_M, \mathcal{U})$ and decryption-key queries under $(\ell_K, \mathcal{U})$, the equalities

$$\sum_{\text{pk} \in \mathcal{H} \cap \mathcal{U} \setminus \text{pk}^*} x_{\text{pk}}^{b, \top} \cdot y_{\text{pk}}^b + x_{\text{pk}^*}^{b, \tau_M \top} \cdot y_{\text{pk}^*}^{b, \tau_K} = \sum_{\text{pk} \in \mathcal{H} \cap \mathcal{U} \setminus \text{pk}^*} x_{\text{pk}}^{0, \top} \cdot y_{\text{pk}}^0 + x_{\text{pk}^*}^{0, \tau_M \top} \cdot y_{\text{pk}^*}^{0, \tau_K}$$

holds for each $\text{pk}^* \in \mathcal{H} \cap \mathcal{U}$ and each pair of index (τ_M, τ_K) that numerates repetitions of encryption queries under $(\text{pk}^*, \ell_M, \mathcal{U})$ and repetitions of decryption-key queries under $(\text{pk}^*, \ell_K, \mathcal{U})$ respectively. Then one has the value

$$\Delta_{\text{pk}^*}^b := x_{\text{pk}^*}^{0, 1 \top} \cdot y_{\text{pk}^*}^{0, 1} - x_{\text{pk}^*}^{b, 1 \top} \cdot y_{\text{pk}^*}^{b, 1} = x_{\text{pk}^*}^{0, \tau_M \top} \cdot y_{\text{pk}^*}^{0, \tau_K} - x_{\text{pk}^*}^{b, \tau_M \top} \cdot y_{\text{pk}^*}^{b, \tau_K}$$

be an invariance across (τ_M, τ_K) . Furthermore, each variance is a share of zero, i.e. $\sum_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \Delta_{\text{pk}}^b = 0$.

The supporting lemma for the adaptive security of DDFE is shown below. At a high level, this lemma provides an information-theoretically secure masking technique for messages in DDFE.

Lemma 2 (FH-IP-DDFE: Injection Lemma). *Considering the security game of DDFE (in Definition 2) for function-hiding inner products, given any challenge bit b , any user set $\mathcal{U}$, any $\mathcal{Q}_K$ -set of key tuples $\{\mathbf{q}_{\ell_K} = (y_{\text{pk}, \ell_K}^{0, \tau_K}, y_{\text{pk}, \ell_K}^{1, \tau_K}, \mathcal{U}, \ell_K)_{\tau_K \in [\text{rep}_{\ell_K}], \text{pk} \in \mathcal{H} \cap \mathcal{U}}\}_{\ell_K \in \mathcal{Q}_K}$ and any message tuple $\mathbf{q}_{\ell_M} = (x_{\text{pk}}^0, x_{\text{pk}}^1, \mathcal{U}, \ell_M)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ that come from admissible queries with respect to Condition (*), then the outputs of experiments $\text{Exp}_{\text{INJ}}^\alpha(b, \mathcal{U}, \mathbf{q}_{\ell_M}, \{\mathbf{q}_{\ell_K}\}_{\ell_K \in \mathcal{Q}_K})$ for $\alpha \in \{0, 1\}$ defined in Figure 7 are perfectly indistinguishable.*

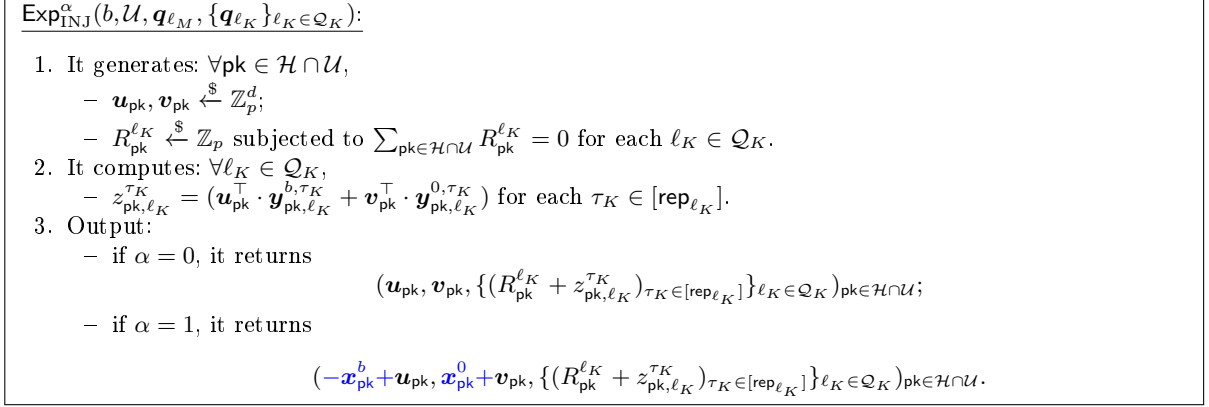


Fig. 7. Experiment description in Injection Lemma. The injected elements are highlighted in blue.

Proof. From Remark 4, given each pk and ℓ_K , one has an invariance $\Delta_{\text{pk}, \ell_K}^b$ that is equal to $(\mathbf{x}_{\text{pk}}^{0\top} \cdot \mathbf{y}_{\text{pk}, \ell_K}^{\tau_K, 0} - \mathbf{x}_{\text{pk}}^{b\top} \cdot \mathbf{y}_{\text{pk}, \ell_K}^{\tau_K, b})$ for every $\tau_K \in [\text{rep}_{\ell_K}]$. In the case $\alpha = 0$, new variables are set as follows

$$\begin{aligned}
 \mathbf{u}'_{\text{pk}} &:= \mathbf{x}_{\text{pk}}^b + \mathbf{u}_{\text{pk}}; & \mathbf{v}'_{\text{pk}} &:= -\mathbf{x}_{\text{pk}}^0 + \mathbf{v}_{\text{pk}}; & R'_{\text{pk}}^{\ell_K} &:= R_{\text{pk}}^{\ell_K} + \Delta_{\text{pk}, \ell_K}^b; \\
 z'_{\text{pk}, \ell_K}{}^{\tau_K} &:= \mathbf{u}'_{\text{pk}}{}^\top \cdot \mathbf{y}_{\text{pk}}^{b, \tau_K} + \mathbf{v}'_{\text{pk}}{}^\top \cdot \mathbf{y}_{\text{pk}}^{0, \tau_K} = z_{\text{pk}, \ell_K}^{\tau_K} - (\mathbf{x}_{\text{pk}}^{0\top} \cdot \mathbf{y}_{\text{pk}, \ell_K}^{\tau_K, 0} - \mathbf{x}_{\text{pk}}^{b\top} \cdot \mathbf{y}_{\text{pk}, \ell_K}^{\tau_K, b}) \\
 &= z_{\text{pk}, \ell_K}^{\tau_K} - \Delta_{\text{pk}, \ell_K}^b.
 \end{aligned}$$

Then the output of the experiment when $\alpha = 0$ can be rewritten as

$$(-\mathbf{x}_{\text{pk}}^b + \mathbf{u}'_{\text{pk}}, \mathbf{x}_{\text{pk}}^0 + \mathbf{v}'_{\text{pk}}, \{(R'_{\text{pk}}^{\ell_K} + z'_{\text{pk}, \ell_K}{}^{\tau_K})_{\tau_K \in [\text{rep}_{\ell_K}]}\}_{\ell_K \in \mathcal{Q}_K})_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$$

Since $(\Delta_{\text{pk}, \ell_K}^b)_{\text{pk}}$ are zero shares, then $(R'_{\text{pk}}^{\ell_K})_{\text{pk}}$ are also uniformly-random zero shares. The lemma is then completed as the rewritten output is of the same distribution as the output in the experiment $\alpha = 1$. $\square$

4.2 FH-IP-DDFE Construction

For every client, let d be an inner-product dimension, let X and Y be a message bound and a function bound of size $\text{poly}(\lambda)$ respectively, let $\mathcal{L}_M$ and $\mathcal{L}_K$ be a message-label space and key-label space respectively. The scheme is described in Figure 8 with the following primitives:

- IPE = (iSetup, iKeyGen, iEnc, iDKGen, iDec) be a single-input function-hiding IPFE;
- UZS = (uSetup, uKeyGen, uSeedGen, uTokGen, uSeedUpt, uShareEval) be a bilinear-updatable pseudorandom zero-sharing scheme over $\mathbb{G}_2$ for a seeding label space $\mathcal{L}_K$ and an updating label space $\mathcal{L}_M$.
- AoNE = (aSetup, aKeyGen, aEnc, aDKGen, aDec) be an all-or-nothing encapsulation scheme.

Remark 5 (Size of Ciphertext/Decryption Key). In the above FH-IP-DDFE construction, if one uses the AoNE scheme that is constructed from a rate-1 identity-based encryption and employed in the hybrid-encryption mode as in [CDSG⁺20], then the size of each DDFE ciphertext/decryption key will be $O_\lambda(d + |\mathcal{U}|)$.

Correctness. By the correctness of the AoNE scheme, one can always recover IPE ciphertexts $(\text{ict}_{\text{pk}})_{\text{pk} \in \mathcal{U}}$ and decryption keys $(\text{idk}_{\text{pk}})_{\text{pk} \in \mathcal{U}}$. The correctness is then implied by the correctness of

Construction:

- **Setup**(1^λ): It generates $\mathcal{PG} \leftarrow \text{PGGen}(1^\lambda)$ and sets up parameters of the underlying schemes: $\text{ipp} \leftarrow \text{iSetup}(1^\lambda)$; $\text{upp} \leftarrow \text{uSetup}(1^\lambda)$; $\text{app} \leftarrow \text{aSetup}(1^\lambda)$. It returns

$$\text{pp} = (\mathcal{PG}, \text{ipp}, \text{upp}, \text{app}).$$

The parameters pp are implicit to other algorithms.

- **KeyGen**($\cdot$): Each client samples

$$k_{\text{pk}} \xleftarrow{\$} \mathcal{K}_{\text{PRF}}; \quad (\text{upk}, \text{usk}_{\text{pk}}) \leftarrow \text{uKeyGen}(\cdot); \quad (\text{apk}, \text{ask}_{\text{pk}}) \leftarrow \text{aKeyGen}(\cdot).$$

It returns $\text{pk} = (\text{upk}, \text{apk})$ and $\text{sk}_{\text{pk}} = (k_{\text{pk}}, \text{usk}_{\text{pk}}, \text{ask}_{\text{pk}})$.

- **Enc**(sk_{pk}, m): It parses $m = (\mathbf{m}, \mathcal{U}_M, \ell_M)$ and computes
 1. a UZS token: $\mathbf{b}_{\text{pk}, \mathcal{U}_M}^{\ell_M} \leftarrow \text{uTokGen}(\text{usk}_{\text{pk}}, (\mathcal{U}_M, \ell_M))$;
 2. a random coin for IPE key generation: $\text{coin}_{\text{pk}} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}_M)$;
 3. a $6d + |\mathcal{U}_M|$ -length IPE secret key: $\text{isk}_{\text{pk}} = \text{iKeyGen}(1^{6d+|\mathcal{U}_M|}, \text{coin}_{\text{pk}})$;
 4. an IPE encryption: $\mathbf{x} = (\mathbf{m}, \mathbf{0}^d)$

$$\text{ict}_{\text{pk}} \leftarrow \text{iEnc}(\text{isk}_{\text{pk}}, \llbracket \mathbf{x}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}_M}^{\ell_M}, 0 \rrbracket_1);$$

5. an AoNE layer on ict_{pk} : $\text{act}_{\text{pk}} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{ict}_{\text{pk}}, \mathcal{U}_M, \ell_M || "ct"))^a$.

It returns the ciphertext

$$\text{ct}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}_M, \ell_M).$$

- **DKGen**(sk_{pk}, k): It parses $k = (\mathbf{k}, \mathcal{U}_K, \ell_K)$ and computes
 1. a UZS seed: $\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}_K}^{\ell_K} \rrbracket_2 \leftarrow \text{uTokGen}(\text{usk}_{\text{pk}}, (\mathcal{U}_K, \ell_K))$;
 2. a random coin for IPE key generation: $\text{coin}_{\text{pk}} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}_K)$;
 3. a $6d + |\mathcal{U}_K|$ -length IPE secret key: $\text{isk}_{\text{pk}} = \text{iKeyGen}(1^{6d+|\mathcal{U}_K|}, \text{coin}_{\text{pk}})$;
 4. an IPE decryption key: $\mathbf{y} = (\mathbf{k}, \mathbf{0}^d)$

$$\text{idk}_{\text{pk}} \leftarrow \text{iDKGen}(\text{isk}_{\text{pk}}, \llbracket \mathbf{y}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{a}_{\text{pk}, \mathcal{U}_K}^{\ell_K}, 0 \rrbracket_2);$$

5. an AoNE layer on idk_{pk} : $\text{act}_{\text{pk}} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{idk}_{\text{pk}}, \mathcal{U}_K, \ell_K || "dk"))$.

It returns the decryption key

$$\text{dk}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}_K, \ell_K).$$

- **Dec**((dk_{pk}) $_{\text{pk} \in \mathcal{U}_K}$, (ct_{pk}) $_{\text{pk} \in \mathcal{U}_M}$, ($\mathcal{U}_M, \ell_M$), ($\mathcal{U}_K, \ell_K$)): If $\mathcal{U}_M = \mathcal{U}_K = \mathcal{U}$ is not true, it returns $\perp$. Otherwise,
 1. it parses $\text{dk}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}, \ell_K)$ and recovers the IPE decryption keys

$$(\text{idk}_{\text{pk}})_{\text{pk} \in \mathcal{U}} = \text{aDec}((\text{act}_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \mathcal{U}, \ell_K || "dk");$$

2. it parses $\text{ct}_{\text{pk}} = (\text{act}'_{\text{pk}}, \mathcal{U}, \ell_M)$ and recovers the IPE ciphertexts

$$(\text{ict}_{\text{pk}})_{\text{pk} \in \mathcal{U}} = \text{aDec}((\text{act}'_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \mathcal{U}, \ell_M || "ct");$$

3. it computes $\llbracket \alpha \rrbracket_T = \sum_{\text{pk} \in \mathcal{U}} \text{iDec}(\text{ict}_{\text{pk}}, \text{idk}_{\text{pk}})$ and returns α .

^a The AoNE scheme can be instantiated separately using two distinct strings: "ct" for ciphertexts and "dk" for functional keys.

Fig. 8. DDFE for Function-Hiding Inner Products

the IPE scheme and the UZS scheme:

$$\begin{aligned}
\sum_{\text{pk} \in \mathcal{U}} \text{iDec}(\text{ict}_{\text{pk}}, \text{idk}_{\text{pk}}) &= \sum_{\text{pk} \in \mathcal{U}} \llbracket \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} + \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K} \rrbracket_T \\
&= \llbracket \sum_{\text{pk} \in \mathcal{U}} \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} \rrbracket_T + e \left(\llbracket 1 \rrbracket_1, \sum_{\text{pk} \in \mathcal{U}} \text{uShareEval}(\text{uSeedUpt}(\text{seed}_{\text{pk}, \mathcal{U}}^{\ell_K}, \text{token}_{\text{pk}, \mathcal{U}}^{\ell_M})) \right) \\
&= \llbracket \sum_{\text{pk} \in \mathcal{U}} \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} \rrbracket_T + e \left(\llbracket 1 \rrbracket_1, \sum_{\text{pk} \in \mathcal{U}} \text{share}_{\text{pk}, \mathcal{U}}^{\ell_K || \ell_M} \right) \\
&= \llbracket \sum_{\text{pk} \in \mathcal{U}} \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} \rrbracket_T + e(\llbracket 1 \rrbracket_1, \llbracket 0 \rrbracket_2) = \llbracket \sum_{\text{pk} \in \mathcal{U}} \mathbf{x}_{\text{pk}}^\top \cdot \mathbf{y}_{\text{pk}} \rrbracket_T = \llbracket \sum_{\text{pk} \in \mathcal{U}} \mathbf{m}_{\text{pk}}^\top \cdot \mathbf{k}_{\text{pk}} \rrbracket_T.
\end{aligned}$$

As the inner product $\sum_{\text{pk} \in \mathcal{U}} \mathbf{m}_{\text{pk}}^\top \cdot \mathbf{k}_{\text{pk}}$ is of size $\text{poly}(\lambda)$, it can always be recovered. $\square$

4.3 Security Analysis

Theorem 2 (Indistinguishability for FH-IP-DDFE). *If IPE is a single-input sym-IND-secure FE for function-hiding inner products, AoNE is a sym-IND-secure all-or-nothing encapsulation, and UZS is an otu-sta-IND-secure updatable pseudorandom zero sharing, then the FH-IP-DDFE scheme constructed in Figure 8 is sta-sym-IND secure (as in Definition 2) in the standard model.*

Proof. In the static-corruption game, we can fix $\mathcal{PK}$ to be the set of parties generated by $\text{QNewHon}()$ queries, $\mathcal{C}$ to be the set of corrupted parties in $\mathcal{PK}$ and $\mathcal{H} = \mathcal{PK} \setminus \mathcal{C}$ to be the set of honest parties. Let q_{xx} be the number of xx-oracle queries where $\text{xx} \in \{\text{QNewHon}, \text{QEnc}, \text{QDKGen}, \text{QCor}\}$. Given $\lambda \in \mathbb{N}$, we denote by $\text{Adv}_{\mathbf{G}_i}$ the advantage of an PPT adversary $\mathcal{A}$ in each game $\mathbf{G}_i$, and Adv_{xx} be the best advantage of any PPT adversary against the primitive xx that is setup with λ .

The DDFE scheme can be parsed as a scheme for messages and keys in the forms $\mathbf{x}^b = (\mathbf{m}^b, \mathbf{0}^d)$ and $\mathbf{y}^b = (\mathbf{k}^b, \mathbf{0}^d)$ respectively. Let $\mathcal{Q}_M$ and $\mathcal{Q}_K$ ⁷ be the set of adaptive encryption queries and decryption key queries sent by $\mathcal{A}$ respectively by the end of the game. When $\text{pk} \in \mathcal{H}$, a decryption key query $(\text{pk}, \mathbf{y}^0, \mathbf{y}^1, \mathcal{U}_K, \ell_K)$ ⁸ $\in \mathcal{Q}_K$ is said to be incomplete if there exists $\text{pk}' \in \mathcal{H} \cap \mathcal{U}_K$ such that there is no decryption key query $(\text{pk}', \mathbf{y}'^0, \mathbf{y}'^1, \mathcal{U}_K, \ell_K) \in \mathcal{Q}_K$. Similarly, when $\text{pk} \in \mathcal{H}$, an encryption query $(\text{pk}, \mathbf{x}^0, \mathbf{x}^1, \mathcal{U}_M, \ell_M) \in \mathcal{Q}_M$ is said to be incomplete if there exists $\text{pk}' \in \mathcal{H} \cap \mathcal{U}_M$ such that there is no encryption query $(\text{pk}', \mathbf{x}'^0, \mathbf{x}'^1, \mathcal{U}_M, \ell_M) \in \mathcal{Q}_M$.

We proceed via a global hybrid argument: we describe the changes in the IND game by using the games $\mathbf{G}_0$, $\mathbf{G}_1$, and $\mathbf{G}_3$ (see Figure 13). Notably, the game $\mathbf{G}_0$ corresponds to sta-sym-IND security game as defined in Definition 2, and the game $\mathbf{G}_3$ corresponds to the case where adversary's advantage is 0 since there is no challenge bit b . We first prove the indistinguishability under the assumption that all encryption queries and decryption-key queries are complete.

The case of complete queries. The transition between $\mathbf{G}_1$ and $\mathbf{G}_3$ requires intermediate games $\mathbf{G}_{1.1}$, $\{\mathbf{G}_{1.1.\ell_M}\}_{\ell_M \in \mathcal{Q}_M}$, and $\mathbf{G}_2$ (see Figure 9). The transition between $\{\mathbf{G}_{1.1.\ell_M}\}_{\ell_M \in \mathcal{Q}_M}$ requires intermediate $(\mathbf{G}_{\ell_M.i}^*)_{i \in [6]}$ (see Figure 10) for each ℓ_M .

The security notion of UZS applies to the transition from $\mathbf{G}_{\ell_M.2}^*$ to $\mathbf{G}_{\ell_M.3}^*$ as in the lemma below.

Lemma 3 (FH-IP-DDFE: Transition from $\mathbf{G}_{\ell_M.2}^*$ to $\mathbf{G}_{\ell_M.3}^*$). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is*

$$\left| \text{Adv}_{\mathbf{G}_{\ell_M.2}^*} - \text{Adv}_{\mathbf{G}_{\ell_M.3}^*} \right| \leq \text{Adv}_{\text{UZS}}^{\text{otu-sta}}.$$

Proof. We build an adversary $\mathcal{B}$ against the otu-sta-IND security of UZS from an adversary $\mathcal{A}$ that distinguishes between two games in the transition. To simulate a FH-IP-DDFE challenger, $\mathcal{B}$ uses the UZS oracles to handle all UZS related operations in DDFE.

⁷ $\mathcal{Q}_M$ and $\mathcal{Q}_K$ contain elements of the form $(\text{pk}, \cdot, \cdot, \mathcal{U}, \ell)$. For a string $\text{xx} \in \{0, 1\}^*$, we denote by $\text{xx} \in \mathcal{Q}_M$ or $\text{xx} \in \mathcal{Q}_K$ if there exists a query containing xx .

⁸ For ease of exposition, we omit subscript/superscript indexes in contexts where these indexes are explicitly mentioned in queries.

Game	iEnc	iKeyGen	Assumption
$\mathbf{G}_1$	$(\mathbf{x}_{\text{pk}}^{b, \tau_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, 0)$	$(\mathbf{y}_{\text{pk}}^{b, \tau_K}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, 0)$	
$\mathbf{G}_{1.1}$	$(\mathbf{x}_{\text{pk}}^{b, \tau_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, 0)$	$(\mathbf{y}_{\text{pk}}^{b, \tau_K}, \mathbf{y}_{\text{pk}}^{0, \tau_K}, \mathbf{0}^{2d}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, 0)$	IND of IPE
$\mathbf{G}_{1.1. \ell_M}$	$(\mathbf{0}^{2d}, \mathbf{x}_{\text{pk}}^{0, \tau_M}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell'_M}, 0)$	same as in	explained
$\ell_M \in \mathcal{Q}_M$	$(\mathbf{x}_{\text{pk}}^{b, \tau_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell'_M}, 0)$	$\mathbf{G}_{2.1}$	in Figure 10
$\mathbf{G}_2$	$(\mathbf{0}^{2d}, \mathbf{x}_{\text{pk}}^{0, \tau_M}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, 0)$	$(\mathbf{0}^{2d}, \mathbf{y}_{\text{pk}}^{0, \tau_K}, \mathbf{0}^{2d}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, 0)$	IND of IPE
$\mathbf{G}_3$	$(\mathbf{x}_{\text{pk}}^{0, \tau_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, 0)$	$(\mathbf{y}_{\text{pk}}^{0, \tau_K}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, 0)$	IND of IPE

Fig. 9. Sequence of hybrids for transition from $\mathbf{G}_1$ to $\mathbf{G}_3$ in the security proof of FH-IP-DDFE (see Theorem 2), under the assumption that all queries are *complete*. All changes are made within iEnc and iDKGen algorithms for the replies of encryption and decryption key generation oracles respectively, and highlighted in blue.

- For each encryption query on $(\text{pk}, \mathbf{x}_{\text{pk}}^0, \mathbf{x}_{\text{pk}}^1, \mathcal{U}, \ell'_M)$,
 - If $\ell'_M \neq \ell_M$: $\mathcal{B}$ obtains $\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell'_M} \leftarrow \text{QTokGen}(\text{pk}, \mathcal{U}, \ell'_M)$ to complete the message to IPE encryption.
 - If $\ell'_M = \ell_M$: it does not have to obtain $\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell'_M}$ as the message to IPE encryption is $(\mathbf{x}_{\text{pk}}^b + \mathbf{u}_{\text{pk}, \mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{0}^{|\mathcal{U}|-1}, 1)$ in this case.
- For each decryption-key query on $(\text{pk}, \mathbf{y}_{\text{pk}}^0, \mathbf{y}_{\text{pk}}^1, \mathcal{U}, \ell_K)$,
 - $\mathcal{B}$ obtains $[\mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}]_2 \leftarrow \text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_K)$ and $([R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K}]_2)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \leftarrow \text{QShare}(\mathcal{U}, \ell_K | \ell_M)$;
 - $\mathcal{B}$ completes the IPE key in $\mathbb{G}_2$ as $[\mathbf{y}_{\text{pk}}^b, \mathbf{y}_{\text{pk}}^0, \mathbf{y}_{\text{pk}}^1, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}, R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K} + z_{\text{pk}, \ell_K}^{\tau_K}]_2$.

The admissibility condition (*) of UZS holds since

- all the corruption queries in FH-IP-DDFE are sent in one shot;
- $\text{QShare}(\mathcal{U}, \ell_K | \ell_M)$ queries are made for the same ℓ_M on every $\mathcal{U}$ and there are no $\text{QTokGen}(\text{pk}, \mathcal{U}, \ell_M)$ queries required.

When $\text{UZS}.b = 0$, one has $[R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K}]_2 = [\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} \top \cdot \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}]_2$ which corresponds to $\mathbf{G}_{\ell_M.2}^*$; and when $\text{UZS}.b = 1$, one has $([R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K}]_2)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ are random shares of $-\sum_{\text{pk} \in \mathcal{C} \cap \mathcal{U}} [\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} \top \cdot \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}]_2$, which corresponds to $\mathbf{G}_{\ell_M.3}^*$. Therefore, one has

$$\left| \text{Adv}_{\mathbf{G}_{\ell_M.2}^*} - \text{Adv}_{\mathbf{G}_{\ell_M.3}^*} \right| \leq \text{Adv}_{\text{UZS}}^{\text{otu-sta}}.$$

□

The Injection Lemma applies to the transition from $\mathbf{G}_{\ell_M.4}^*$ to $\mathbf{G}_{\ell_M.5}^*$ as in the lemma below.

Lemma 4 (FH-IP-DDFE: Transition from $\mathbf{G}_{\ell_M.4}^*$ to $\mathbf{G}_{\ell_M.5}^*$). *The two games $\mathbf{G}_{\ell_M.4}^*$ and $\mathbf{G}_{\ell_M.5}^*$ are identical.*

Proof. In these games, we can parse $R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_K} = R_{\text{pk}, \mathcal{U}}^{0, \ell_M, \ell_K} + R_{\text{pk}, \mathcal{U}}^{1, \ell_M, \ell_K}$ where $(R_{\text{pk}, \mathcal{U}}^{1, \ell_M, \ell_K})_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ are random shares of $-\sum_{\text{pk} \in \mathcal{C} \cap \mathcal{U}} \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} \top \cdot \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_K}$ and $(R_{\text{pk}, \mathcal{U}}^{0, \ell_M, \ell_K})_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ are random shares of 0.

Given each $\mathcal{U}$, we set $\mathbf{q}_{\ell_M} = (\mathbf{x}_{\text{pk}}^{0, 1}, \mathbf{x}_{\text{pk}}^{1, 1}, \mathcal{U}, \ell_M)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}}$ and for each $\ell_K \in \mathcal{Q}_K$, we set $\mathbf{q}_{\ell_K} = (\mathbf{y}_{\text{pk}}^{0, \tau_K}, \mathbf{y}_{\text{pk}}^{1, \tau_K}, \mathcal{U}, \ell_K)_{\tau_K \in [\text{rep}_{\ell_K}], \text{pk} \in \mathcal{H} \cap \mathcal{U}}$. Then the difference between two games are the outputs of $\{\text{Exp}_{\text{INJ}}^\alpha(b, \mathcal{U}, \mathbf{q}_{\ell_M}, \{\mathbf{q}_{\ell_K}\}_{\ell_K \in \mathcal{Q}_K})\}_{\mathcal{U} \in \mathcal{Q}_M}$, where $\alpha = 0$ corresponds to $\mathbf{G}_{\ell_M.4}^*$ and $\alpha = 1$ corresponds to $\mathbf{G}_{\ell_M.5}^*$. By Lemma 2, we have $\text{Exp}_{\text{INJ}}^0(b, \mathcal{U}, \mathbf{q}_{\ell_M}, \{\mathbf{q}_{\ell_K}\}_{\ell_K \in \mathcal{Q}_K}) = \text{Exp}_{\text{INJ}}^1(b, \mathcal{U}, \mathbf{q}_{\ell_M}, \{\mathbf{q}_{\ell_K}\}_{\ell_K \in \mathcal{Q}_K})$ with respect to experiment randomness $(\mathbf{u}_{\text{pk}, \mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk}, \mathcal{U}}^{\ell_M}, R_{\text{pk}, \mathcal{U}}^{0, \ell_M, \ell_K})$ that are generated independently for each $\mathcal{U}$. Therefore, the lemma is complete. □

Lemmas for other transitions and the indistinguishability in the case of incomplete queries are provided in Section B.2. By completing the proof in both cases, we complete the theorem. □

Game	Adjustment	Assumption
$\mathbf{G}_{\ell_M}^* := \mathbf{G}_{1.1.\ell_M}$	as in Figure 9	
$\mathbf{G}_{\ell_M.1}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{x}_{\text{pk}}^{b,\tau_M} + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{b}_{\text{pk},\mathcal{U}}^{\ell_M}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : on all labels ℓ_K , $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, -\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, 0)$	IND of IPE
$\mathbf{G}_{\ell_M.2}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{x}_{\text{pk}}^{b,\tau_M} + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{0}^{ U -1}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : on all labels ℓ_K , $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, \mathbf{b}_{\text{pk},\mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K} + z_{\text{pk},\ell_K}^{\tau_K})$ where $z_{\text{pk},\ell_K}^{\tau_K} = -\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{y}_{\text{pk}}^{b,\tau_K}$	IND of IPE
$\mathbf{G}_{\ell_M.3}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{x}_{\text{pk}}^{b,\tau_M} + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{0}^{ U -1}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : on all labels ℓ_K , $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, \mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K} + z_{\text{pk},\ell_K}^{\tau_K})$ where $(\mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K})_{\text{pk} \in \mathcal{H} \cup \mathcal{U}}$ are random subjected to $\sum_{\text{pk} \in \mathcal{H} \cup \mathcal{U}} \mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K} = -\sum_{\text{pk} \in \mathcal{C} \cup \mathcal{U}} \mathbf{b}_{\text{pk},\mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}$	IND of UZS
$\mathbf{G}_{\ell_M.4}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{x}_{\text{pk}}^{b,\tau_M} + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{ U -1}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : on all labels ℓ_K , $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, \mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K} + z_{\text{pk},\ell_K}^{\tau_K})$ where $z_{\text{pk},\ell_K}^{\tau_K} = -(\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{y}_{\text{pk}}^{b,\tau_K} + \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{y}_{\text{pk}}^{0,\tau_K})$	IND of IPE
$\mathbf{G}_{\ell_M.5}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{x}_{\text{pk}}^{b,\tau_M} - \mathbf{x}_{\text{pk}}^{b,1} + \mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{x}_{\text{pk}}^{0,1} + \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{ U -1}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, \mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K} + z_{\text{pk},\ell_K}^{\tau_K})$	Statistical by Injection Lemma
$\mathbf{G}_{\ell_M.6}^*$	<u>iEnc</u> : on label ℓ_M , $(\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{2d}, \mathbf{x}_{\text{pk}}^{0,\tau_M} + \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{0}^{ U -1}, 1)$ where $\mathbf{u}_{\text{pk},\mathcal{U}}^{\ell_M}, \mathbf{v}_{\text{pk},\mathcal{U}}^{\ell_M} \xleftarrow{\$} \mathbb{Z}_p^d$ <u>iKeyGen</u> : $(\mathbf{y}_{\text{pk}}^{b,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{y}_{\text{pk}}^{0,\tau_K}, \mathbf{a}_{\text{pk},\mathcal{U}}^{\ell_K}, \mathbf{R}_{\text{pk},\mathcal{U}}^{\ell_M,\ell_K} + z_{\text{pk},\ell_K}^{\tau_K})$	IND of IPE
$\mathbf{G}_{\ell_M+1}^* := \mathbf{G}_{1.1.\ell_M+1}$	as in Figure 9	Symmetric transitions as from $\mathbf{G}_{\ell_M}^*$ to $\mathbf{G}_{\ell_M.4}^*$

Fig. 10. Sequence of hybrids for each transition from $\mathbf{G}_{1.1.\ell_M}$ to $\mathbf{G}_{1.1.\ell_M+1}$ in Figure 9. We denote by $\mathcal{H}$ (and $\mathcal{C}$) the set of honest (and corrupted) users and $(\ell_M + 1)$ the subsequent message label of ℓ_M in the ordered set $\mathcal{Q}_M$ of encryption queries.

5 Attribute-Weighted-Sum DDFE

In this section, we construct a DDFE scheme for attribute-weighted sums that is sel-sta-sym-IND secure in the standard model.

5.1 Construction

Let $\mathcal{L}_M$ and $\mathcal{L}_K$ be a message-label space and a key-label space respectively. The AWS-DDFE scheme is described in Figure 11 with the following primitives: AWIPE = (aiSetup, aiKeyGen, aiEnc, aiDKGen, aiDec) be a FE for attribute-weighted sums with function-hiding inner products; UZS be a bilinear-updatable pseudorandom zero-sharing scheme over $\mathbb{G}_2$ for a seeding-label space $\mathcal{L}_K$ and an updating-label space $\mathcal{L}_M$; AoNE be an all-or-nothing encapsulation scheme.

We defer the correctness and the efficiency analysis to Section C.

5.2 Security Analysis

Theorem 3 (Indistinguishability for AWS-DDFE). *If AWIPE is a single-input sel-sym-IND-secure FE for attribute-weighted sums with function-hiding inner products, AoNE is a sel-sym-IND-secure all-or-nothing encapsulation, and UZS is an otu-sta-IND-secure updatable pseudorandom zero sharing, then the AWS-DDFE scheme constructed in Figure 11 is sel-sta-sym-IND secure (as in Definition 2) in the standard model.*

We defer the proof of the above theorem to Section C.

Acknowledgements. We would like to thank Ky Nguyen, Duong Hieu Phan and David Pointcheval for fruitful discussions that motivated this work. We also appreciate the anonymous reviewers for their insightful feedback and suggestions. This work was supported in part by the France 2030 ANR Project ANR-22-PECY-003 SecureCompute.

References

- ABDP15. M. Abdalla, F. Bourse, A. De Caro, and D. Pointcheval. Simple functional encryption schemes for inner products. In *PKC 2015: 18th International Conference on Theory and Practice of Public Key Cryptography, Lecture Notes in Computer Science* 9020, pages 733–751, Gaithersburg, MD, USA, March 30 – April 1, 2015. Springer, Heidelberg, Germany.
- ABG19. M. Abdalla, F. Benhamouda, and R. Gay. From single-input to multi-client inner-product functional encryption. In *Advances in Cryptology – ASIACRYPT 2019, Part III, Lecture Notes in Computer Science* 11923, pages 552–582, Kobe, Japan, December 8–12, 2019. Springer, Heidelberg, Germany.
- ABKW19. M. Abdalla, F. Benhamouda, M. Kohlweiss, and H. Waldner. Decentralizing inner-product functional encryption. In *PKC 2019: 22nd International Conference on Theory and Practice of Public Key Cryptography, Part II, Lecture Notes in Computer Science* 11443, pages 128–157, Beijing, China, April 14–17, 2019. Springer, Heidelberg, Germany.
- ABM⁺20. M. Abdalla, F. Bourse, H. Marival, D. Pointcheval, A. Soleimanian, and H. Waldner. Multi-client inner-product functional encryption in the random-oracle model. In *SCN 20: 12th International Conference on Security in Communication Networks, Lecture Notes in Computer Science* 12238, pages 525–545, Amalfi, Italy, September 14–16, 2020. Springer, Heidelberg, Germany.
- ACF⁺18. M. Abdalla, D. Catalano, D. Fiore, R. Gay, and B. Ursu. Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings. In *Advances in Cryptology – CRYPTO 2018, Part I, Lecture Notes in Computer Science* 10991, pages 597–627, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Heidelberg, Germany.
- ACF⁺20. S. Agrawal, M. Clear, O. Frieder, S. Garg, A. O’Neill, and J. Thaler. Ad hoc multi-input functional encryption. In *ITCS 2020: 11th Innovations in Theoretical Computer Science Conference*, pages 40:1–40:41, Seattle, WA, USA, January 12–14, 2020. LIPIcs.
- ACGU20. M. Abdalla, D. Catalano, R. Gay, and B. Ursu. Inner-product functional encryption with fine-grained access control. In *Advances in Cryptology – ASIACRYPT 2020, Part III, Lecture Notes in Computer Science* 12493, pages 467–497, Daejeon, South Korea, December 7–11, 2020. Springer, Heidelberg, Germany.

Construction:

- **Setup**(1^λ): It generates $\mathcal{PG} \leftarrow \text{PGGen}(1^\lambda)$ and sets up parameters of the underlying schemes: $\text{aipp} \leftarrow \text{aiSetup}(1^\lambda)$; $\text{upp} \leftarrow \text{uSetup}(1^\lambda)$; $\text{app} \leftarrow \text{aSetup}(1^\lambda)$. It returns

$$\text{pp} = (\mathcal{PG}, \text{aipp}, \text{upp}, \text{app}).$$

The parameters pp are implicit to other algorithms.

- **KeyGen**($\cdot$): Each client samples

$$k_{\text{pk}} \xleftarrow{\$} \mathcal{K}_{\text{PRF}}; \quad (\text{upk}, \text{usk}_{\text{pk}}) \leftarrow \text{uKeyGen}(\cdot); \quad (\text{apk}, \text{ask}_{\text{pk}}) \leftarrow \text{aKeyGen}(\cdot).$$

It returns $\text{pk} = (\text{upk}, \text{apk})$ and $\text{sk}_{\text{pk}} = (k_{\text{pk}}, \text{usk}_{\text{pk}}, \text{ask}_{\text{pk}})$.

- **Enc**(sk_{pk}, m): It parses $m = ((\mathbf{x}_j, \mathbf{z}_j)_{j \in [N]}, \mathcal{U}_M, \ell_M)$ ^a and computes

1. a UZS token: $\mathbf{b}_{\text{pk}, \mathcal{U}_M}^{\ell_M} \leftarrow \text{uTokGen}(\text{usk}_{\text{pk}}, \mathcal{U}_M, \ell_M)$;
2. a random coin for AWIPE key generation: $\text{coin}_{\text{pk}} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}_M)$;
3. a AWIPE secret key: $\text{aisk}_{\text{pk}} = \text{aiKeyGen}(1_{\text{ip}}^{|\mathcal{U}_M|}; \text{coin}_{\text{pk}})$;
4. an AWIPE encryption:

$$\text{aict}_{\text{pk}} \leftarrow \text{aiEnc}(\text{aisk}_{\text{pk}}, (\mathbf{x}_j, \mathbf{z}_j)_{j \in [N]}, \llbracket \mathbf{b}_{\text{pk}, \mathcal{U}_M}^{\ell_M}, 0 \rrbracket_1);$$

5. an AoNE layer on aict_{pk} : $\text{act}_{\text{pk}} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{aict}_{\text{pk}}, \mathcal{U}_M, \ell_M || "ct"))$ ^b.

It returns the ciphertext

$$\text{ct}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}_M, \ell_M).$$

- **DKGen**(sk_{pk}, k): It parses $k = (\mathbf{f} := (f_{\text{pk}}, \text{pk})_{\text{pk} \in \mathcal{U}_K}, \mathcal{U}_K)$ and computes

1. a UZS seed: $\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}_K}^{\ell_f} \rrbracket_2 \leftarrow \text{uSeedGen}(\text{usk}_{\text{pk}}, \mathcal{U}_K, \ell_f)$ ^c;
2. a random coin for AWIPE key generation: $\text{coin}_{\text{pk}} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}_K)$;
3. a AWIPE secret key: $\text{aisk}_{\text{pk}} = \text{aiKeyGen}(1_{\text{ip}}^{|\mathcal{U}_K|}; \text{coin}_{\text{pk}})$;
4. an AWIPE decryption key:

$$\text{aidk}_{\text{pk}} \leftarrow \text{aiDKGen}(\text{aisk}_{\text{pk}}, f_{\text{pk}}, \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}_K}^{\ell_f}, 0 \rrbracket_2);$$

5. an AoNE layer on aidk_{pk} : $\text{act}_{\text{pk}} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{aidk}_{\text{pk}}, \mathcal{U}_K, \ell_f || "dk"))$.

It returns the decryption key

$$\text{dk}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}_K, \ell_f).$$

- **Dec**((dk_{pk}) $_{\text{pk} \in \mathcal{U}_K}$, (ct_{pk}) $_{\text{pk} \in \mathcal{U}_M}$, ($\mathcal{U}_M, \ell_M$), ($\mathcal{U}_K, \ell_f$)): If $\mathcal{U}_M = \mathcal{U}_K = \mathcal{U}$ is not true, it returns $\perp$. Otherwise,

1. it parses $\text{dk}_{\text{pk}} = (\text{act}_{\text{pk}}, \mathcal{U}, \ell_f)$ and recovers the AWIPE decryption keys

$$(\text{aidk}_{\text{pk}})_{\text{pk} \in \mathcal{U}} = \text{aDec}((\text{act}_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \mathcal{U}, \ell_f || "dk");$$

2. it parses $\text{ct}_{\text{pk}} = (\text{act}'_{\text{pk}}, \mathcal{U}, \ell_M)$ and recovers the AWIPE ciphertexts

$$(\text{aict}_{\text{pk}})_{\text{pk} \in \mathcal{U}} = \text{aDec}((\text{act}'_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \mathcal{U}, \ell_M || "ct");$$

It returns $\llbracket \alpha \rrbracket_T = \sum_{\text{pk} \in \mathcal{U}} \text{aiDec}(\text{aict}_{\text{pk}}, \text{aidk}_{\text{pk}})$.

^a Each client can choose an arbitrary number N of AWS inputs.

^b The AoNE scheme can be instantiated separately using two distinct strings: "ct" for ciphertexts and "dk" for functional keys.

^c $\ell_f \in \mathcal{L}_K$ contains a description of $\mathbf{f}$.

Fig. 11. DDFE for Attribute-Weighted Sums

- AGT21a. S. Agrawal, R. Goyal, and J. Tomida. Multi-input quadratic functional encryption from pairings. In *Advances in Cryptology – CRYPTO 2021, Part IV, Lecture Notes in Computer Science* 12828, pages 208–238, Virtual Event, August 16–20, 2021. Springer, Heidelberg, Germany.
- AGT21b. S. Agrawal, R. Goyal, and J. Tomida. Multi-party functional encryption. In *TCC 2021: 19th Theory of Cryptography Conference, Part II, Lecture Notes in Computer Science* 13043, pages 224–255, Raleigh, NC, USA, November 8–11, 2021. Springer, Heidelberg, Germany.
- AGT22. S. Agrawal, R. Goyal, and J. Tomida. Multi-input quadratic functional encryption: Stronger security, broader functionality. In *TCC 2022: 20th Theory of Cryptography Conference, Part I, Lecture Notes in Computer Science* 13747, pages 711–740, Chicago, IL, USA, November 7–10, 2022. Springer, Heidelberg, Germany.
- AGW20. M. Abdalla, J. Gong, and H. Wee. Functional encryption for attribute-weighted sums from k -Lin. In *Advances in Cryptology – CRYPTO 2020, Part I, Lecture Notes in Computer Science* 12170, pages 685–716, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.
- ALMT20. S. Agrawal, B. Libert, M. Maitra, and R. Titu. Adaptive simulation security for inner product functional encryption. In *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part I, Lecture Notes in Computer Science* 12110, pages 34–64, Edinburgh, UK, May 4–7, 2020. Springer, Heidelberg, Germany.
- ALS16. S. Agrawal, B. Libert, and D. Stehlé. Fully secure functional encryption for inner products, from standard assumptions. In *Advances in Cryptology – CRYPTO 2016, Part III, Lecture Notes in Computer Science* 9816, pages 333–362, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Heidelberg, Germany.
- ATY23. S. Agrawal, J. Tomida, and A. Yadav. Attribute-based multi-input FE (and more) for attribute-weighted sums. In *Advances in Cryptology – CRYPTO 2023, Part IV, Lecture Notes in Computer Science* 14084, pages 464–497, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Heidelberg, Germany.
- BCFG17. C. E. Z. Baltico, D. Catalano, D. Fiore, and R. Gay. Practical functional encryption for quadratic functions with applications to predicate encryption. In *Advances in Cryptology – CRYPTO 2017, Part I, Lecture Notes in Computer Science* 10401, pages 67–98, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Heidelberg, Germany.
- BIK⁺17. K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 1175–1191, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- BJK15. A. Bishop, A. Jain, and L. Kowalczyk. Function-hiding inner product encryption. In *Advances in Cryptology – ASIACRYPT 2015, Part I, Lecture Notes in Computer Science* 9452, pages 470–491, Auckland, New Zealand, November 30 – December 3, 2015. Springer, Heidelberg, Germany.
- BSW11. D. Boneh, A. Sahai, and B. Waters. Functional encryption: Definitions and challenges. In *TCC 2011: 8th Theory of Cryptography Conference, Lecture Notes in Computer Science* 6597, pages 253–273, Providence, RI, USA, March 28–30, 2011. Springer, Heidelberg, Germany.
- CDG⁺18. J. Chotard, E. Dufour Sans, R. Gay, D. H. Phan, and D. Pointcheval. Decentralized multi-client functional encryption for inner product. In *Advances in Cryptology – ASIACRYPT 2018, Part II, Lecture Notes in Computer Science* 11273, pages 703–732, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Heidelberg, Germany.
- CDSG⁺20. J. Chotard, E. Dufour-Sans, R. Gay, D. H. Phan, and D. Pointcheval. Dynamic decentralized functional encryption. In *Advances in Cryptology – CRYPTO 2020, Part I, Lecture Notes in Computer Science* 12170, pages 747–775, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.
- Cha07. M. Chase. Multi-authority attribute based encryption. In *TCC 2007: 4th Theory of Cryptography Conference, Lecture Notes in Computer Science* 4392, pages 515–534, Amsterdam, The Netherlands, February 21–24, 2007. Springer, Heidelberg, Germany.
- CLT18. G. Castagnos, F. Laguillaumie, and I. Tucker. Practical fully secure unrestricted inner product functional encryption modulo p . In *Advances in Cryptology – ASIACRYPT 2018, Part II, Lecture Notes in Computer Science* 11273, pages 733–764, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Heidelberg, Germany.
- DP19. E. Dufour Sans and D. Pointcheval. Unbounded inner-product functional encryption with succinct keys. In *ACNS 19: 17th International Conference on Applied Cryptography and Network Security, Lecture Notes in Computer Science* 11464, pages 426–441, Bogota, Colombia, June 5–7, 2019. Springer, Heidelberg, Germany.
- GGG⁺14. S. Goldwasser, S. D. Gordon, V. Goyal, A. Jain, J. Katz, F.-H. Liu, A. Sahai, E. Shi, and H.-S. Zhou. Multi-input functional encryption. In *Advances in Cryptology – EUROCRYPT 2014, Lecture Notes in Computer Science* 8441, pages 578–602, Copenhagen, Denmark, May 11–15, 2014. Springer, Heidelberg, Germany.

- GKL⁺13. S. D. Gordon, J. Katz, F.-H. Liu, E. Shi, and H.-S. Zhou. Multi-input functional encryption. Cryptology ePrint Archive, Report 2013/774, 2013. <https://eprint.iacr.org/2013/774>.
- IW14. Y. Ishai and H. Wee. Partial garbling schemes and their applications. In *ICALP 2014: 41st International Colloquium on Automata, Languages and Programming, Part I, Lecture Notes in Computer Science* 8572, pages 650–662, Copenhagen, Denmark, July 8–11, 2014. Springer, Heidelberg, Germany.
- KDK11. K. Kursawe, G. Danezis, and M. Kohlweiss. Privacy-friendly aggregation for the smart-grid. In *PETS 2011: 11th International Symposium on Privacy Enhancing Technologies, Lecture Notes in Computer Science* 6794, pages 175–191, Waterloo, ON, Canada, July 27–29, 2011. Springer, Heidelberg, Germany.
- LT19. B. Libert and R. Titu. Multi-client functional encryption for linear functions in the standard model from LWE. In *Advances in Cryptology – ASIACRYPT 2019, Part III, Lecture Notes in Computer Science* 11923, pages 520–551, Kobe, Japan, December 8–12, 2019. Springer, Heidelberg, Germany.
- LW11. A. B. Lewko and B. Waters. Decentralizing attribute-based encryption. In *Advances in Cryptology – EUROCRYPT 2011, Lecture Notes in Computer Science* 6632, pages 568–588, Tallinn, Estonia, May 15–19, 2011. Springer, Heidelberg, Germany.
- LWG⁺23. Y. Li, J. Wei, F. Guo, W. Susilo, and X. Chen. Robust decentralized multi-client functional encryption: Motivation, definition, and inner-product constructions. In *Advances in Cryptology – ASIACRYPT 2023, Part V, Lecture Notes in Computer Science* 14442, pages 134–165, Guangzhou, China, December 4–8, 2023. Springer, Heidelberg, Germany.
- MJ18. Y. Michalevsky and M. Joye. Decentralized policy-hiding ABE with receiver privacy. In *ESORICS 2018: 23rd European Symposium on Research in Computer Security, Part II, Lecture Notes in Computer Science* 11099, pages 548–567, Barcelona, Spain, September 3–7, 2018. Springer, Heidelberg, Germany.
- MKMS22. J. M. B. Mera, A. Karmakar, T. Marc, and A. Soleimanian. Efficient lattice-based inner-product functional encryption. In *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II, Lecture Notes in Computer Science* 13178, pages 163–193, Virtual Event, March 8–11, 2022. Springer, Heidelberg, Germany.
- NPP22. K. Nguyen, D. H. Phan, and D. Pointcheval. Multi-client functional encryption with fine-grained access control. In *Advances in Cryptology – ASIACRYPT 2022, Part I, Lecture Notes in Computer Science* 13791, pages 95–125, Taipei, Taiwan, December 5–9, 2022. Springer, Heidelberg, Germany.
- NPP23a. D. D. Nguyen, D. H. Phan, and D. Pointcheval. Verifiable decentralized multi-client functional encryption for inner product. In *Advances in Cryptology – ASIACRYPT 2023, Part V, Lecture Notes in Computer Science* 14442, pages 33–65, Guangzhou, China, December 4–8, 2023. Springer, Heidelberg, Germany.
- NPP23b. K. Nguyen, D. H. Phan, and D. Pointcheval. Optimal security notion for decentralized multi-client functional encryption. In *ACNS 23: 21st International Conference on Applied Cryptography and Network Security, Part II, Lecture Notes in Computer Science* 13906, pages 336–365, Kyoto, Japan, June 19–22, 2023. Springer, Heidelberg, Germany.
- NPS24. K. Nguyen, D. Pointcheval, and R. Schädlich. Decentralized multi-client functional encryption with strong security. *IACR Communications in Cryptology*, 1(2), 2024.
- SV23. E. Shi and N. Vanjani. Multi-client inner product encryption: Function-hiding instantiations without random oracles. In *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I, Lecture Notes in Computer Science* 13940, pages 622–651, Atlanta, GA, USA, May 7–10, 2023. Springer, Heidelberg, Germany.
- SW05. A. Sahai and B. R. Waters. Fuzzy identity-based encryption. In *Advances in Cryptology – EUROCRYPT 2005, Lecture Notes in Computer Science* 3494, pages 457–473, Aarhus, Denmark, May 22–26, 2005. Springer, Heidelberg, Germany.
- Tom19. J. Tomida. Tightly secure inner product functional encryption: Multi-input and function-hiding constructions. In *Advances in Cryptology – ASIACRYPT 2019, Part III, Lecture Notes in Computer Science* 11923, pages 459–488, Kobe, Japan, December 8–12, 2019. Springer, Heidelberg, Germany.
- TT18. J. Tomida and K. Takashima. Unbounded inner product functional encryption from bilinear maps. In *Advances in Cryptology – ASIACRYPT 2018, Part II, Lecture Notes in Computer Science* 11273, pages 609–639, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Heidelberg, Germany.
- Üna20. A. Ünal. Impossibility results for lattice-based functional encryption schemes. In *Advances in Cryptology – EUROCRYPT 2020, Part I, Lecture Notes in Computer Science* 12105, pages 169–199, Zagreb, Croatia, May 10–14, 2020. Springer, Heidelberg, Germany.
- ZLZ⁺24. Z. Zhu, J. Li, K. Zhang, J. Gong, and H. Qian. Registered functional encryptions from pairings. Cryptology ePrint Archive, Paper 2024/327, 2024. <https://eprint.iacr.org/2024/327>.

A More Definitions

A.1 Prime Order Group.

Let GGen be a prime-order group generator, a probabilistic polynomial time (PPT) algorithm that on input the security parameter 1^λ returns a description $\mathcal{G} = (\mathbb{G}, p, P)$ of an additive cyclic group $\mathbb{G}$ of order p for a 2λ -bit prime p , whose generator is P . For $a \in \mathbb{Z}_p$, define $\llbracket a \rrbracket = aP \in \mathbb{G}$ as the *implicit representation* of a in $\mathbb{G}$.

From a random element $\llbracket a \rrbracket \in \mathbb{G}$, it is computationally hard to compute the value a (the discrete logarithm problem). Given $\llbracket a \rrbracket, \llbracket b \rrbracket \in \mathbb{G}$ and a scalar $x \in \mathbb{Z}_p$, one can efficiently compute $\llbracket ax \rrbracket \in \mathbb{G}$ and $\llbracket a + b \rrbracket = \llbracket a \rrbracket + \llbracket b \rrbracket \in \mathbb{G}$.

Definition 9 (Decisional Diffie-Hellman Assumption). *The Decisional Diffie-Hellman Assumption states that, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,*

$$\text{Adv}_{\text{DDH}}(\mathcal{A}) := \left| \mathbb{P} \left[\mathcal{A}(\mathcal{G}, \mathcal{D}_b) = b \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, \mathcal{G} \xleftarrow{\$} \text{GGen}(1^\lambda) \\ a, r, s \xleftarrow{\$} \mathbb{Z}_p, d_0 = ar, d_1 = s \\ \mathcal{D}_b = (\llbracket a \rrbracket, \llbracket r \rrbracket, \llbracket d_b \rrbracket) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

Definition 10 (m -Multi DDH Assumption [CDG⁺18]). *For all $\lambda \in \mathbb{N}$ and for every PPT adversary $\mathcal{A}$ running within time t , then*

$$\text{Adv}_{m\text{-DDH}}(\mathcal{A}, t) := \left| \mathbb{P} \left[\mathcal{A}(\mathcal{G}, \mathcal{D}_b) = b \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, \mathcal{G} \xleftarrow{\$} \text{GGen}(1^\lambda) \\ X, Y_j, Z_j \xleftarrow{\$} \mathbb{G} \forall j \in [m] \\ \mathcal{D}_0 = (X, (Y_j, \text{CDH}(X, Y_j))_{j=1}^m) \\ \mathcal{D}_1 = (X, (Y_j, Z_j)_{j=1}^m) \end{array} \right] - \frac{1}{2} \right|$$

is bounded by $\text{Adv}_{\text{DDH}}(\mathcal{A}, t + 4m \times t_{\mathbb{G}})$, where $t_{\mathbb{G}}$ is the time for an exponentiation in $\mathbb{G}$.

A.2 Pairing Group.

Let PGen be a pairing group generator, a PPT algorithm that on input the security parameter 1^λ returns a description $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, P_1, P_2, e)$ of asymmetric pairing groups where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are additive cyclic groups of order p for a 2λ -bit prime p , P_1 and P_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable (non-degenerate) bilinear group elements. For $s \in \{1, 2, T\}$ and $a \in \mathbb{Z}_p$, we define $\llbracket a \rrbracket_s = aP_s \in \mathbb{G}_s$ as the implicit representation of a in $\mathbb{G}_s$, and then for any $0 < B < \frac{p}{2}$, we define $\mathbb{G}_s^{[-B, B]} = \{\llbracket a \rrbracket_s \in \mathbb{G}_s : a \in [-B, B]\}$. Given $\llbracket a \rrbracket_1, \llbracket b \rrbracket_2$, one can efficiently compute $\llbracket ab \rrbracket_T$ using the pairing e .

Definition 11 (Symmetric eXternal Diffie-Hellman Assumption). *The Symmetric eXternal Diffie-Hellman (SXDH) Assumption states that, in a pairing group $\mathcal{PG} \xleftarrow{\$} \text{PGen}(1^\lambda)$, the DDH assumption holds in both $\mathbb{G}_1$ and $\mathbb{G}_2$.*

A.3 Arithmetic Branching Programs

Definition 12 (Arithmetic Branching Programs (ABPs) [IW14, AGW20, ATY23]). *An arithmetic branching program $f : \mathbb{Z}_p^{n_0} \rightarrow \mathbb{Z}_p$ is defined by a prime p , a directed acyclic graph (V, E) , two special vertices $v_0, v_1 \in V$, and a labeling function $\sigma : E \rightarrow \mathcal{F}^{\text{Affine}}$, where $\mathcal{F}^{\text{Affine}}$ consists of all affine functions $g : \mathbb{Z}_p^{n_0} \rightarrow \mathbb{Z}_p$. The size of f is the number of vertices $|V|$. Given an input $\mathbf{x} \in \mathbb{Z}_p^{n_0}$ to the ABP, we can assign a $\mathbb{Z}_p$ element to edge $e \in E$ by $\sigma(e)(\mathbf{x})$. Let P be the set of all paths from v_0 to v_1 . Each element in P can be represented by a subset of E . The output of the ABP on input $\mathbf{x}$ is defined as $\sum_{E' \in P} \prod_{e \in E'} \sigma(e)(\mathbf{x})$. We can extend the definition of ABPs for functions $f : \mathbb{Z}_p^{n_0} \rightarrow \mathbb{Z}_p^{n_1}$ by evaluating each output in a coordinate-wise manner and denote such a function class by $\mathcal{F}_{n_0, n_1}^{\text{ABP}}$.*

There exists a linear-time algorithm that converts any boolean formula, boolean branching program or arithmetic formula to an arithmetic branching program with a constant blow-up in the representation size, so ABPs can be considered as a stronger computational model than the others.

Definition 13 (All-or-Nothing Encapsulation [CDSG+20]). AoNE is defined on messages of length L and label space $\mathcal{L}$ as follows:

$$\mathcal{K} = \emptyset \quad \mathcal{M} = \{0, 1\}^L \times \mathcal{S}(\mathcal{PK}) \times \mathcal{L}$$

Then, $F(\epsilon_K, (\text{pk}, (x_{\text{pk}}, \mathcal{U}_{\text{pk}}, \ell_{\text{pk}}))_{\text{pk} \in \mathcal{U}}) = (\mathcal{U}_{\text{pk}}, \ell_{\text{pk}})_{\text{pk} \in \mathcal{U}}$ for any $\mathcal{U} \in \mathcal{L}(\mathcal{PK})$ and

$$F(\epsilon_K, (\text{pk}, m_{\text{pk}})_{\text{pk} \in \mathcal{U}_M}) = \begin{cases} (\text{pk}, x_{\text{pk}})_{\text{pk} \in \mathcal{U}_M} & \text{if condition } (*) \\ \perp & \text{otherwise.} \end{cases}$$

and AoNE condition $(*)$ is: $\exists \ell \in \mathcal{L}, \forall \text{pk} \in \mathcal{U}_M, m_{\text{pk}} = (x_{\text{pk}}, \mathcal{U}_M, \ell)$.

We note that a concrete AoNE construction that is secure in the standard model is provided in [CDSG+20].

A.4 Pseudorandom Functions (PRF)

Definition 14 (PRF). A PRF from input space $\mathcal{X}$ to output space $\mathcal{Y}$ is secure if for any security parameter $\lambda \in \mathbb{N}$, and for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\text{Adv}_{\text{PRF}, \mathcal{A}}(\lambda) := \left| \mathbb{P} \left[\mathcal{A}^{\mathcal{O}_{\text{PRF}}^b(\cdot)}(1^\lambda) = b \mid \begin{array}{l} K \xleftarrow{\$} \{0, 1\}^\lambda, b \xleftarrow{\$} \{0, 1\} \\ \forall \ell \in \mathcal{X} : \\ \mathcal{O}_{\text{PRF}}^0(\ell) := \text{PRF}_K(\ell) \\ \mathcal{O}_{\text{PRF}}^1(\ell) := \text{RF}(\ell) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}_{\text{PRF}}^b(\cdot)$ is an oracle depending on the challenge bit b and RF is a random function computed on the fly.

A.5 Non-Interactive Key Exchange (NIKE)

Definition 15 (Non-Interactive Key Exchange). A NIKE scheme consists of three PPT algorithms:

- **SetUp**(λ): On input a security parameter λ , it outputs public parameters pp . Those parameters are implicit arguments to all the other algorithms;
- **KeyGen**($\cdot$): It generates and outputs a party's public key $\text{pk} \in \mathcal{PK}$ and the corresponding secret key sk_{pk} ;
- **SharedKey**($\text{pk}, \text{sk}_{\text{pk}'}$): On input a public key and a secret key corresponding to a different public key, it deterministically outputs a shared key K .

Correctness. For all security parameters $\lambda \in \mathbb{N}$, it holds that

$$\Pr[\text{SharedKey}(\text{pk}, \text{sk}_{\text{pk}'}) = \text{SharedKey}(\text{pk}', \text{sk}_{\text{pk}})] = 1,$$

where the probability is taken over $\text{pp} \leftarrow \text{SetUp}(\lambda)$, $(\text{pk}, \text{sk}_{\text{pk}}) \leftarrow \text{KeyGen}()$, $(\text{pk}', \text{sk}_{\text{pk}'}) \leftarrow \text{KeyGen}()$.

Definition 16 (Security for NIKE). No adversary $\mathcal{A}$ should be able to win the following security game against a challenger $\mathcal{C}$, with unlimited and adaptive access to the oracles QNewHon , QRev , QTest , and QCor described below:

- *Initialize:* the challenger runs the setup algorithm $\text{pp} \leftarrow \text{SetUp}(\lambda)$ and chooses a random bit $b \xleftarrow{\$} \{0, 1\}$. It initializes the set $\mathcal{H}$ of honest participants to $\emptyset$. It provides pp to the adversary $\mathcal{A}$;
- *Participant creation queries* $\text{QNewHon}()$: it generates $(\text{pk}, \text{sk}_{\text{pk}}) \leftarrow \text{KeyGen}()$, stores the association $(\text{pk}, \text{sk}_{\text{pk}})$ in the set $\mathcal{H}$ of honest keys, and returns pk to the adversary;

- *Reveal queries* $\text{QRev}(\text{pk}, \text{pk}')$: it requires at least one of pk and pk' be in $\mathcal{H}$. Assume $\text{pk} \in \mathcal{H}$, then it returns $\text{SharedKey}(\text{pk}', \text{sk}_{\text{pk}})$;
- *Test queries* $\text{QTest}(\text{pk}, \text{pk}')$: it requires that both pk and pk' were generated via QNewHon .
 - if $b = 0$, it returns $\text{SharedKey}(\text{pk}', \text{sk}_{\text{pk}})$;
 - if $b = 1$, it returns a (uniformly) random value and stores the value so it can consistently answer further queries to $\text{QTest}(\text{pk}, \text{pk}')$ or $\text{QTest}(\text{pk}', \text{pk})$.
- *Corruption queries* $\text{QCor}(\text{pk})$: it recovers the secret key sk associated to pk from $\mathcal{H}$ and returns it, then removes (pk, sk) from $\mathcal{H}$. If pk is not associated with any secret key (i.e. it is not in $\mathcal{H}$), then nothing is returned;
- *Finalize*: $\mathcal{A}$ provides its guess b' on the bit b , and this procedure outputs the result β according to the analysis given below.

Finalize outputs the bit $\beta = (b' = b)$ unless a QCor query was made for any public key which was involved in a query to QTest , or a QRev query was made for a pair of public keys which was also involved in a QTest query, in which case a random bit β is returned. We say **NIKE** is secure if given any parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, the following holds:

$$\text{Adv}_{\text{NIKE}}(\mathcal{A}) = |\Pr[\beta = 1] - 1/2| \leq \text{negl}(\lambda).$$

B Deferred Proofs

B.1 Updatable Zero Sharing

Lemma 5 (UZS: Transition from $\mathbf{G}_0$ to $\mathbf{G}_1$). For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is

$$|\text{Adv}_0 - \text{Adv}_1| \leq \text{Adv}_{\text{NIKE}}.$$

Proof. We build an adversary $\mathcal{B}$ against the IND security of **NIKE** from an adversary $\mathcal{A}$ that distinguishes between $\mathbf{G}_0$ and $\mathbf{G}_1$. To simulate a UZS challenger, $\mathcal{B}$ uses the **NIKE** oracles as follows:

- For every $\text{QNewHon}()$ query, $\mathcal{B}$ returns pk from the **NIKE**. $\text{QNewHon}()$ query to $\mathcal{A}$.
- For every $\text{QCor}(\text{pk})$ query, $\mathcal{B}$ obtains $\text{NIKE.sk}_{\text{pk}}$ from the **NIKE**. $\text{QCor}(\text{pk})$ query, and computes $k_{\text{pk}, \text{pk}'} \leftarrow \text{NIKE.SharedKey}(\text{pk}', \text{NIKE.sk}_{\text{pk}})$ for all $\text{pk}' \in \mathcal{PK}$ to complete the reply to $\mathcal{A}$.
- Instead of generating by itself the keys $(k_{\text{pk}, \text{pk}'})_{(\text{pk}, \text{pk}') \in \mathcal{H}^2, \text{pk} \neq \text{pk}'}$, the adversary $\mathcal{B}$ uses the challenge shared keys $k_{\text{pk}, \text{pk}'}$ from the **NIKE**. $\text{QTest}(\text{pk}, \text{pk}')$ queries.
- $\mathcal{B}$ outputs $\mathcal{A}$'s guess for the challenge bit **NIKE**. b .

The admissibility condition (*) of **NIKE** holds since the set of corrupted users in UZS is sent in one shot. Therefore, when **NIKE**. $b = 0$, $\mathcal{B}$ is playing $\mathbf{G}_0$; when **NIKE**. $b = 1$, $\mathcal{B}$ is playing $\mathbf{G}_1$. $\square$

Lemma 6 (UZS: Transition from $\mathbf{G}_1$ to $\mathbf{G}_2$). For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is

$$|\text{Adv}_1 - \text{Adv}_2| \leq \left(\frac{q_{\text{QNewHon}} - q_{\text{QCor}}}{2} \right) \cdot \text{Adv}_{\text{PRF}}.$$

Proof. We proceed by using multiple hybrid games over each pair of different honest parties $(\text{pk}, \text{pk}') \in \mathcal{H}^2$. For each transition, we build an adversary $\mathcal{B}$ against the IND security of PRF from an adversary $\mathcal{A}$ that distinguishes two games in the transition. To simulate a UZS challenger, $\mathcal{B}$ uses the PRF oracle, instead of generating by itself the PRF key $k_{\text{pk}, \text{pk}'}$ to handle all $\text{PRF}_{k_{\text{pk}, \text{pk}'}}$ related operations, and finally outputs $\mathcal{A}$'s guess for the challenge bit $\text{PRF}.b$. Since the number of honest pairs is bounded by $\left(\frac{q_{\text{QNewHon}} - q_{\text{QCor}}}{2} \right)$, one completes the proof. $\square$

B.2 Function-Hiding Inner-Product DDFE

The case of complete queries. The deferred lemmas for transitions are provided below.

Lemma 7 (FH-IP-DDFE: Transition from $\mathbf{G}_0$ to $\mathbf{G}_1$). For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is

$$|\text{Adv}_{\mathbf{G}_0} - \text{Adv}_{\mathbf{G}_1}| \leq (q_{\text{QNewHon}} - q_{\text{QCor}}) \cdot \text{Adv}_{\text{PRF}}.$$

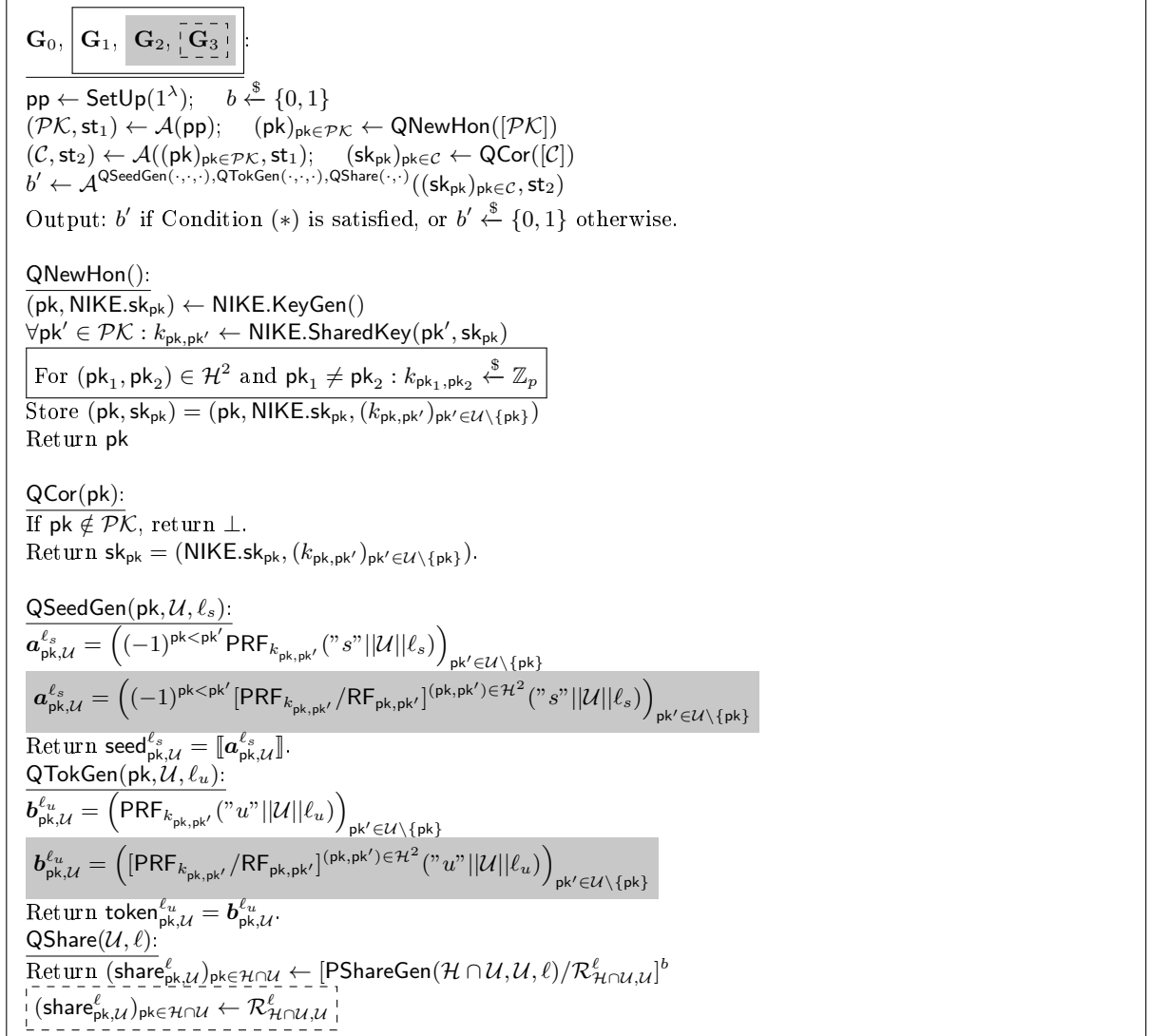


Fig. 12. Games for the otu-sta-IND proof of UZS in Theorem 1

$\mathbf{G}_0, \boxed{\mathbf{G}_1 \begin{smallmatrix} \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \end{smallmatrix} \mathbf{G}_3 \begin{smallmatrix} \text{---} \\ \text{---} \\ \text{---} \end{smallmatrix}};$

$\text{pp} \leftarrow \text{SetUp}(1^\lambda); \quad b \xleftarrow{\$} \{0, 1\}$
 $(\mathcal{PK}, \text{st}_1) \leftarrow \mathcal{A}(\text{pp}); \quad (\text{pk})_{\text{pk} \in \mathcal{PK}} \leftarrow \text{QNewHon}([\mathcal{PK}])$
 $(\mathcal{C}, \text{st}_2) \leftarrow \mathcal{A}((\text{pk})_{\text{pk} \in \mathcal{PK}}, \text{st}_1); \quad (\text{sk}_{\text{pk}})_{\text{pk} \in \mathcal{C}} \leftarrow \text{QCor}([\mathcal{C}])$
 $b' \leftarrow \mathcal{A}^{\text{QEnc}(\cdot, \cdot, \cdot, \cdot), \text{QDKGen}(\cdot, \cdot, \cdot, \cdot)}((\text{sk}_{\text{pk}})_{\text{pk} \in \mathcal{C}}, \text{st}_2)$
 Output b' if Condition (*) is satisfied, or $b' \leftarrow \{0, 1\}$ otherwise.

QNewHon():
 $k_{\text{pk}} \leftarrow \mathcal{K}_{\text{PRF}}; \quad (\text{upk}, \text{usk}_{\text{pk}}) \leftarrow \text{uKeyGen}(); \quad (\text{apk}, \text{ask}_{\text{pk}}) \leftarrow \text{aKeyGen}()$
 $\text{pk} = (\text{upk}, \text{apk}); \quad \text{sk}_{\text{pk}} = (k_{\text{pk}}, \text{usk}_{\text{pk}}, \text{ask}_{\text{pk}})$
 Store $(\text{pk}, \text{sk}_{\text{pk}})$ and return pk .

QDKGen($\text{pk}, \mathbf{y}^0 = (\mathbf{k}^0, \mathbf{0}^d), \mathbf{y}^1 = (\mathbf{k}^1, \mathbf{0}^d), (\mathcal{U}, \ell)$):
 $\llbracket \mathbf{a} \rrbracket_2 \leftarrow \text{uSeedGen}(\text{usk}_{\text{pk}}, (\mathcal{U}, \ell))$
 $\text{coin} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}) \quad \boxed{\text{coin} \leftarrow \text{RF}_{\text{pk}}(\mathcal{U}) \quad \forall \text{pk} \in \mathcal{H}}$
 $\text{isk} \leftarrow \text{iKeyGen}(1^{6d+|\mathcal{U}|}; \text{coin})$
 $\llbracket \mathbf{g} \rrbracket_2 = \llbracket (\mathbf{y}^b, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{a}, 0) \rrbracket_2 \quad \boxed{\llbracket \mathbf{g} \rrbracket_2 = \llbracket (\mathbf{y}^0, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \mathbf{a}, 0) \rrbracket_2}$
 $\text{idk} \leftarrow \text{iDKGen}(\text{isk}, \llbracket \mathbf{g} \rrbracket_2)$
 $\text{act} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{idk}, \mathcal{U}, \ell || "dk"))$
 Return $(\text{act}, \mathcal{U}, \ell)$.

QEnc($\text{pk}, \mathbf{x}^0 = (\mathbf{m}^0, \mathbf{0}^d), \mathbf{x}^1 = (\mathbf{m}^1, \mathbf{0}^d), (\mathcal{U}, \ell)$):
 $\bar{\mathbf{b}} \leftarrow \text{uTokGen}(\text{usk}_{\text{pk}}, (\mathcal{U}, \ell))$
 $\text{coin} \leftarrow \text{PRF}_{k_{\text{pk}}}(\mathcal{U}) \quad \boxed{\text{coin} \leftarrow \text{RF}_{\text{pk}}(\mathcal{U}) \quad \forall \text{pk} \in \mathcal{H}}$
 $\text{isk} \leftarrow \text{iKeyGen}(1^{6d+|\mathcal{U}|}; \text{coin})$
 $\llbracket \mathbf{h} \rrbracket_1 = \llbracket (\mathbf{x}^b, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \bar{\mathbf{b}}, 0) \rrbracket_1 \quad \boxed{\llbracket \mathbf{h} \rrbracket_1 = \llbracket (\mathbf{x}^0, \mathbf{0}^{2d}, \mathbf{0}^{2d}, \bar{\mathbf{b}}, 0) \rrbracket_1}$
 $\text{ict} \leftarrow \text{iEnc}(\text{isk}, \llbracket \mathbf{h} \rrbracket_1)$
 $\text{act} \leftarrow \text{aEnc}(\text{ask}_{\text{pk}}, (\text{ict}_{\text{pk}}, \mathcal{U}, \ell || "ct"))$
 Return $(\text{act}, \mathcal{U}, \ell)$.

QCor(pk):
 If $\text{pk} \notin \mathcal{PK}$, return $\perp$.
 Return $\text{sk}_{\text{pk}} = (k_{\text{pk}}, \text{usk}_{\text{pk}}, \text{ask}_{\text{pk}})$.

Fig. 13. Games for the sta-sym-IND proof of FH-IP-DDFE in Theorem 2

Proof. We proceed by using multiple hybrid games for each $\mathbf{pk} \in \mathcal{H}$. For each transition, we build an adversary $\mathcal{B}$ against the IND security of PRF from an adversary $\mathcal{A}$ that distinguishes two games in the transition. To simulate a FH-IP-DDFE challenger, $\mathcal{B}$ uses the PRF oracle, instead of generating by itself the PRF key $k_{\mathbf{pk}}$ to handle all $\text{PRF}_{k_{\mathbf{pk}}}$ related operations, and finally outputs $\mathcal{A}$'s guess for the challenge bit $\text{PRF}.b$. Since the number of honest parties is bounded by $(q_{\text{QNewHon}} - q_{\text{QCor}})$, one completes the proof. $\square$

Lemma 8 (FH-IP-DDFE: Transitions to $\mathbf{G}_{1.1}$, $\mathbf{G}_{\ell_M.1}^*$, $\mathbf{G}_{\ell_M.2}^*$, $\mathbf{G}_{\ell_M.4}^*$, $\mathbf{G}_{\ell_M.6}^*$, $\mathbf{G}_2$, $\mathbf{G}_3$ from their previous games). For any PPT adversary $\mathcal{A}$, the advantage in distinguishing between any two games is upper-bounded by

$$(q_{\text{QEnc}} + q_{\text{QDKGen}}) \cdot \text{Adv}_{\text{IPE}}^{\text{sym}}.$$

Proof. We proceed by using multiple hybrid games for each pair of honest $(\mathbf{pk}, \mathcal{U}) \in \mathcal{Q}_M \cup \mathcal{Q}_K$. We build an adversary $\mathcal{B}$ against the (adaptive) sym-IND security of IPE from an adversary $\mathcal{A}$ that distinguishes between two games in the considered transition. To simulate a FH-IP-DDFE challenger, $\mathcal{B}$ uses the IPE oracles to handle all IPE related operations for the reply of each $(\mathbf{pk}, \mathcal{U})$ -involved query from $\mathcal{A}$.

- For each complete $\text{QDKGen}(\mathbf{pk}, \mathbf{y}^0, \mathbf{y}^1, \mathcal{U}, \ell_K)$ query, let $\mathbf{g}_{\mathbf{pk}, \mathcal{U}}^0$ and $\mathbf{g}_{\mathbf{pk}, \mathcal{U}}^1$ be the IPE keys that $\mathcal{B}$ needs to prepare in the previous game and the subsequent game respectively. Then $\mathcal{B}$ sends $(\mathbf{g}_{\mathbf{pk}, \mathcal{U}}^0, \mathbf{g}_{\mathbf{pk}, \mathcal{U}}^1)$ to IPE decryption key generation oracle. It uses the returned decryption key $\text{idk}_{\mathbf{pk}}$ to complete the reply to $\mathcal{A}$.
- For each complete $\text{QEnc}(\mathbf{pk}, \mathbf{x}^0, \mathbf{x}^1, \mathcal{U}, \ell_M)$ query, let $\mathbf{h}_{\mathbf{pk}, \mathcal{U}}^0$ and $\mathbf{h}_{\mathbf{pk}, \mathcal{U}}^1$ be the IPE messages that $\mathcal{B}$ needs to prepare in the previous game and the subsequent game respectively. Then $\mathcal{B}$ sends $(\mathbf{h}_{\mathbf{pk}, \mathcal{U}}^0, \mathbf{h}_{\mathbf{pk}, \mathcal{U}}^1)$ to IPE encryption oracle. It uses the returned ciphertext $\text{ict}_{\mathbf{pk}}$ to complete the reply to $\mathcal{A}$.
- $\mathcal{B}$ outputs $\mathcal{A}$'s guess for the challenge bit $\text{IPE}.b$.

The admissibility condition (*) of IPE in each transition holds since one always has $\mathbf{g}_{\mathbf{pk}, \mathcal{U}}^0{}^\top \cdot \mathbf{h}_{\mathbf{pk}, \mathcal{U}}^0 = \mathbf{g}_{\mathbf{pk}, \mathcal{U}}^1{}^\top \cdot \mathbf{h}_{\mathbf{pk}, \mathcal{U}}^1$. Proceeding for each $(\mathbf{pk}, \mathcal{U})$ in queries where $\mathbf{pk} \in \mathcal{H}$, one completes the lemma. $\square$

The case of incomplete queries. We first remark that as all transitions in the case of complete queries do not need to exploit the structure of merged messages and keys, namely $\mathbf{x} = (\mathbf{m}, \mathbf{0}^d)$ and $\mathbf{y} = (\mathbf{k}, \mathbf{0}^d)$, then the indistinguishability of complete queries DDFE also holds for arbitrary $2d$ -length message $\mathbf{x}$ and key $\mathbf{y}$. We use this remark for the following hybrid argument.

Game $\mathbf{G}_1^{\text{in}}$: This game corresponds to $\mathbf{G}_1$, where for each $\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}$, the challenger encrypts $\mathbf{x} = (\mathbf{m}^b, \mathbf{0}^d)$ to answer any query on $(\mathbf{pk}, \mathbf{m}^0, \mathbf{m}^1, \mathcal{U}, \ell_M)$ and generates decryption keys for $\mathbf{y} = (\mathbf{k}^b, \mathbf{0}^d)$ to answer any query on $(\mathbf{pk}, \mathbf{k}^0, \mathbf{k}^1, \mathcal{U}, \ell_K)$.

Game $\mathbf{G}_2^{\text{in}}$: The change is that the challenger encrypts $\mathbf{x} = (\mathbf{m}^b, \mathbf{m}^0)$.

Game $\mathbf{G}_3^{\text{in}}$: The change is that the challenger generates decryption keys for $\mathbf{y} = (\mathbf{0}^d, \mathbf{k}^0)$.

Game $\mathbf{G}_4^{\text{in}}$: The change is that the challenger encrypts $\mathbf{x} = (\mathbf{0}^d, \mathbf{m}^0)$. As $\mathbf{y} = (\mathbf{0}^d, \mathbf{k}^0)$ in this game, there is no more challenge bit b , so the adversary's advantage is 0. which is identical to that in $\mathbf{G}_3$.

Inspired by the proof strategy relying on AoNE and complete-queries DDFE in [SV23], we use the following intermediate games for the transition from $\mathbf{G}_1^{\text{in}}$ to $\mathbf{G}_2^{\text{in}}$. For every $(\mathcal{U}_M, \ell_M) \in \mathcal{Q}_M$,

Games $\mathbf{G}_{1.(\mathcal{U}_M, \ell_M)}^{\text{in}}$: In this game, to answer any query on $(\mathbf{pk}, \mathbf{m}^0, \mathbf{m}^1, \mathcal{U}, \ell)$ the challenger encrypts $\mathbf{x} = (\mathbf{m}^b, \mathbf{m}^0)$ if $(\mathcal{U}, \ell) < (\mathcal{U}_M, \ell_M)$ and $\mathbf{x} = (\mathbf{m}^b, \mathbf{0}^d)$ if $(\mathcal{U}, \ell) \geq (\mathcal{U}_M, \ell_M)$. The only change between games of adjacent pairs $(\mathcal{U}_M, \ell_M)$ and $(\mathcal{U}_M, \ell_M) + 1$ is the selection of message $\mathbf{x}$ to encrypt for encryption queries on $(\mathbf{pk}, \mathbf{m}^0, \mathbf{m}^1, \mathcal{U}_M, \ell_M)$.

An adversary $\mathcal{A}$ that distinguishes between two games can be used by an adversary $\mathcal{B}$ to play against either DDFE or AoNE as follows.

- $\mathcal{B}$ first guesses if $\mathcal{A}$ sends complete queries at $(\mathcal{U}_M, \ell_M)$ or not. If yes, it starts a sta-sym-IND game of DDFE for complete queries. If no, it starts an sym-IND game of AoNE. If in the end the guess is wrong, it aborts the underlying security game. In both games, $\mathcal{B}$ receives encryption queries $(\mathbf{pk}, \mathbf{m}^0, \mathbf{m}^1, \mathcal{U}, \ell)$ from $\mathcal{A}$ and prepares challenge messages and keys as follows:

- $\mathbf{y} = \mathbf{y}^0 = \mathbf{y}^1 = (\mathbf{k}^b, \mathbf{0}^d)$;
 - $\mathbf{x}^0 = \mathbf{x}^1 = (\mathbf{m}^b, \mathbf{m}^0)$ if $(\mathcal{U}, \ell) < (\mathcal{U}_M, \ell_M)$;
 - $\mathbf{x}^0 = \mathbf{x}^1 = (\mathbf{m}^b, \mathbf{0}^d)$ if $(\mathcal{U}, \ell) > (\mathcal{U}_M, \ell_M)$;
 - $\mathbf{x}^0 = (\mathbf{m}^b, \mathbf{0}^d)$ and $\mathbf{x}^1 = (\mathbf{m}^b, \mathbf{m}^0)$ if $(\mathcal{U}, \ell) = (\mathcal{U}_M, \ell_M)$.
- If $\mathcal{B}$ is playing against DDFE, then $\mathcal{B}$ sends the queries $(\mathbf{pk}, \mathbf{x}^0, \mathbf{x}^1, \mathcal{U}, \ell)$ and $(\mathbf{pk}, \mathbf{y}, \mathbf{y}, \mathcal{U}, \ell)$ to DDFE oracles. Note that if encryption queries on $(\mathcal{U}, \ell) \neq (\mathcal{U}_M, \ell_M)$ and decryption-key queries sent by $\mathcal{A}$ are not complete, $\mathcal{B}$ can complete them by sending itself queries for identical messages and keys on missing $\mathbf{pk} \in \mathcal{U} \cap \mathcal{H}$ to DDFE oracles. If $\mathcal{A}$ sends complete queries at $(\mathcal{U}_M, \ell_M)$, $\mathcal{B}$ will be an adversary against DDFE with all complete queries.
- If $\mathcal{B}$ is playing against AoNE, then $\mathcal{B}$ uses IPE to encrypt $(\mathbf{x}^0, \mathbf{x}^1)$ into $(\text{ict}^0, \text{ict}^1)$, and sends $(\mathbf{pk}, \text{ict}^0, \text{ict}^1, \mathcal{U}, \ell || "ct")$ to AoNE encryption oracle. For decryption-key queries, it uses IPE to generate keys idk for $\mathbf{y}$ and sends $(\mathbf{pk}, \text{idk}, \text{idk}, \mathcal{U}, \ell || "dk")$ to AoNE decryption-key oracle. If $\mathcal{A}$ sends incomplete queries at $(\mathcal{U}_M, \ell_M)$, $\mathcal{B}$ will be an admissible adversary against AoNE.

The advantage of $\mathcal{A}$ between each pair of games $\mathbf{G}_{1.(\mathcal{U}_M, \ell_M)}^{\text{in}}$ and $\mathbf{G}_{1.(\mathcal{U}_M, \ell_M)+1}^{\text{in}}$ is then upper-bounded by the maximum of best advantages in complete-queries DDFE and AoNE, which is negligible. We apply a similar strategy of using complete-queries DDFE and AoNE to the transitions between $\mathbf{G}_2^{\text{in}}$, $\mathbf{G}_3^{\text{in}}$, and $\mathbf{G}_4^{\text{in}}$.

C Attribute-Weighted-Sum DDFE

Correctness. From the scheme in Figure 11, one can always recover AWIPE ciphertexts $(\text{aict}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}$ and decryption keys $(\text{aidk}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}$ from AoNE. The correctness is then implied by the correctness of the AWIPE scheme and the UZS scheme:

$$\begin{aligned}
& \sum_{\mathbf{pk} \in \mathcal{U}} \text{aiDec}(\text{aict}_{\mathbf{pk}}, \text{aidk}_{\mathbf{pk}}) \\
&= \sum_{\mathbf{pk} \in \mathcal{U}} \left[\sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk}, j})^\top \cdot \mathbf{z}_{\mathbf{pk}, j} + \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M}{}^\top \cdot \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f} \right]_T \\
&= \left[\sum_{\mathbf{pk} \in \mathcal{U}} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk}, j})^\top \cdot \mathbf{z}_{\mathbf{pk}, j} \right]_T \\
&+ e \left(\llbracket 1 \rrbracket_1, \sum_{\mathbf{pk} \in \mathcal{U}} \text{uShareEval}(\text{uSeedUpt}(\text{seed}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}, \text{token}_{\mathbf{pk}, \mathcal{U}}^{\ell_M})) \right) \\
&= \left[\sum_{\mathbf{pk} \in \mathcal{U}} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk}, j})^\top \cdot \mathbf{z}_{\mathbf{pk}, j} \right]_T + e \left(\llbracket 1 \rrbracket_1, \sum_{\mathbf{pk} \in \mathcal{U}} \text{share}_{\mathbf{pk}, \mathcal{U}}^{\ell_f || \ell_M} \right) \\
&= \left[\sum_{\mathbf{pk} \in \mathcal{U}} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk}, j})^\top \cdot \mathbf{z}_{\mathbf{pk}, j} \right]_T + e(\llbracket 1 \rrbracket_1, \llbracket 0 \rrbracket_2) \\
&= \left[\sum_{\mathbf{pk} \in \mathcal{U}} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk}, j})^\top \cdot \mathbf{z}_{\mathbf{pk}, j} \right]_T.
\end{aligned}$$

□

Remark 6 (Size of Ciphertext/Decryption Key). In the AWS-DDFE construction (Figure 11), if one uses the AWSw/IP-FE that is constructed based on the MDDH_k assumption as in [ATY23] and the AoNE that is constructed from a rate-1 identity-based encryption and employed in the hybrid-encryption mode as in [CDSG+20], then the complexity for the size each DDFE ciphertext will be $O_\lambda(N(kn_0 + n_1 + n))$, and the size of each DDFE decryption key will be $O_\lambda(t(kn_0 + n_1 + n))$. Here (t, n_0, n_1) are parameters involved by ABPs in $\mathcal{F}_{n_0, n_1}^{\text{ABP}}$.

Like FH-IP-DDFE, we have a remark from the admissibility of AWS-DDFE.

Remark 7 (Invariance in AWS-DDFE). Given any admissible adversary against AWS-DDFE and any challenge bit b , Condition (*) in Definition 2 implies that among encryption queries under $(\ell_M, \mathcal{U})$

and decryption-key queries under $(\ell_f, \mathcal{U})$, the equalities

$$\begin{aligned} & \sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U} \setminus \mathbf{pk}^*} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk},j})^\top \mathbf{z}_{\mathbf{pk},j}^b + \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^\tau)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{b,\tau} \\ &= \sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U} \setminus \mathbf{pk}^*} \sum_{j \in [N_{\mathbf{pk}}]} f_{\mathbf{pk}}(\mathbf{x}_{\mathbf{pk},j})^\top \mathbf{z}_{\mathbf{pk},j}^0 + \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^\tau)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{0,\tau} \end{aligned}$$

holds for each $\mathbf{pk}^* \in \mathcal{H} \cap \mathcal{U}$ and each index τ that numerates repetitions of encryption queries under $(\mathbf{pk}^*, \ell_M, \mathcal{U})$. Then one has the value

$$\begin{aligned} \Delta_{\mathbf{pk}^*}^b &:= \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^\tau)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{0,\tau} - \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^\tau)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{b,\tau} \\ &= \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^1)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{0,1} - \sum_{j \in [N_{\mathbf{pk}^*}]} f_{\mathbf{pk}^*}(\mathbf{x}_{\mathbf{pk}^*,j}^1)^\top \mathbf{z}_{\mathbf{pk}^*,j}^{b,1} \end{aligned}$$

be an invariance across τ . Furthermore, each variance is an additive share of zero, i.e. $\sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}} \Delta_{\mathbf{pk}}^b = 0$.

Proof (Theorem 3). This is a selective-static game where any adversary has to send encryption queries $\mathcal{Q}_M$, decryption-key queries $\mathcal{Q}_K$ and corruption queries for $\mathcal{C} = \mathcal{PK} \setminus \mathcal{H}$ in one shot. For brevity, we use the notations $\hat{x} := (\mathbf{x}_j, \mathbf{z}_j)_{j \in [N]}$ and $\hat{f}$ for each ABP function f such that $\hat{f}(\hat{x}) := \sum_{j \in [N]} f(\mathbf{x}_j)^\top \mathbf{z}_j$. We proceed via a hybrid argument: we describe the global changes in the IND game by using the games $\mathbf{G}_0$, $\mathbf{G}_1$, $\mathbf{G}_2$ and $\mathbf{G}_3$ that are described below; the transition between $\mathbf{G}_2$ and $\mathbf{G}_3$ requires intermediate games $(\mathbf{G}_{2.(\mathcal{U}, \ell_M).i})_{i \in [5]}$ (see Figure 14) for each pair $(\mathcal{U}, \ell_M) \in \mathcal{Q}_M$. Notably, the game $\mathbf{G}_0$ corresponds to sel-sta-sym-IND security game, and the game $\mathbf{G}_3$ corresponds to the case where adversary's advantage is 0 since there is no challenge bit b .

Game $\mathbf{G}_1$: The change is that the challenger uses a random function $\text{RF}_{\mathbf{pk}}$ instead of $\text{PRF}_{k_{\mathbf{pk}}}$ for $\mathbf{pk} \in \mathcal{H}$. The indistinguishability is implied by the security of the pseudorandom functions.

Game $\mathbf{G}_2$: When $\mathbf{pk} \in \mathcal{H}$, a decryption key query $(\mathbf{pk}, (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_K}, \mathcal{U}_K) \in \mathcal{Q}_K$ is said to be incomplete if there exists $\mathbf{pk}' \in \mathcal{H} \cap \mathcal{U}_K$ such that there is no key query $(\mathbf{pk}', (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}_K}, \mathcal{U}_K) \in \mathcal{Q}_K$. For that query, $\text{act}_{\mathbf{pk}}$ is changed to the encapsulation of $(0, \mathcal{U}_K, \ell_f \| "dk")$. Similarly, when $\mathbf{pk} \in \mathcal{H}$, an encryption query $(\mathbf{pk}, \hat{x}_{\mathbf{pk}}^0, \hat{x}_{\mathbf{pk}}^1, \mathcal{U}_M, \ell_M) \in \mathcal{Q}_M$ is said to be incomplete if there exists $\mathbf{pk}' \in \mathcal{H} \cap \mathcal{U}_M$ such that there is no encryption query $(\mathbf{pk}', \hat{x}_{\mathbf{pk}'}^0, \hat{x}_{\mathbf{pk}'}^1, \mathcal{U}_M, \ell_M) \in \mathcal{Q}_M$. For that query, $\text{act}_{\mathbf{pk}}$ is changed to the encapsulation of $(0, \mathcal{U}_M, \ell_M \| "ct")$. The indistinguishability is implied by the security of the AoNE scheme.

Game $\mathbf{G}_3$: In this game, for every complete encryption query on the tuple $(\mathbf{pk}, \hat{x}_{\mathbf{pk}}^0, \hat{x}_{\mathbf{pk}}^1, \mathcal{U}_M, \ell_M)$, the challenger chooses $\hat{x}_{\mathbf{pk}}^0$ to encrypt.

Lemmas for transitions between the intermediate games $(\mathbf{G}_{2.(\mathcal{U}, \ell_M).i})_{i \in [5]}$ (see Figure 14) for each pair $(\mathcal{U}, \ell_M) \in \mathcal{Q}_M$ are provided in the following section, which complete the transition from $\mathbf{G}_2$ to $\mathbf{G}_3$. Therefore, the theorem is complete. $\square$

Lemma 9 (AWS-DDFE: Transition from $\mathbf{G}_{2.(\mathcal{U}, \ell_M).0}$ to $\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}$). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is*

$$\left| \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).0}} - \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}} \right| \leq (q_{\text{QNewHon}} - q_{\text{QCor}}) \cdot \text{Adv}_{\text{AWIPE}}^{\text{sel-sym}}.$$

Proof. On a fixed user set $\mathcal{U}$, we proceed by using multiple hybrid games for each $\mathbf{pk} \in \mathcal{H}$. We build an adversary $\mathcal{B}$ against the sel-sym security of AWIPE from an adversary $\mathcal{A}$ that distinguishes between two games in the transition. To simulate a AWS-DDFE challenger, $\mathcal{B}$ uses the AWIPE oracles to handle all AWIPE related operations for the reply of each $(\mathbf{pk}, \mathcal{U})$ -involved query from $\mathcal{A}$.

– For each complete QDKGen($\mathbf{pk}, (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}, \mathcal{U}$) query, $\mathcal{B}$ prepares the AWIPE key as

$$\begin{aligned} k^0 &= (\hat{f}_{\mathbf{pk}}, \llbracket \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}, 0 \rrbracket_2), \\ k^1 &= (\hat{f}_{\mathbf{pk}}, \llbracket \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}, \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f} \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M} \rrbracket_2) \end{aligned}$$

where $\mathbf{f} = (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}$ and sends $(\mathbf{pk}, k^0, k^1)$ to the AWIPE decryption-key oracle. It uses the returned decryption key to complete the reply to $\mathcal{A}$.

Game	Adjustment	Assumption
$\mathbf{G}_{2.(\mathcal{U}, \ell_M).0}$	<u>aiEnc</u> : $(\mathcal{U}', \ell'_M) < (\mathcal{U}, \ell_M): (\hat{x}_{\text{pk}}^0, \mathbf{b}_{\text{pk}, \mathcal{U}'}^{\ell'_M}, 0)$ $(\mathcal{U}', \ell'_M) \geq (\mathcal{U}, \ell_M): (\hat{x}_{\text{pk}}^b, \mathbf{b}_{\text{pk}, \mathcal{U}'}^{\ell'_M}, 0)$ <u>aiDKGen</u> : same as in $\mathbf{G}_2$	Hybrids on $(\mathcal{U}', \ell'_M) < (\mathcal{U}, \ell_M)$
$\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}$	<u>aiEnc</u> : $(\mathcal{U}', \ell'_M) = (\mathcal{U}, \ell_M): (\hat{x}_{\text{pk}}^b, \mathbf{0}^{ U -1}, 1)$ <u>aiKeyGen</u> : $\mathcal{U}' = \mathcal{U}: (\hat{f}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f \top} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M})$	IND of AWIPE
$\mathbf{G}_{2.(\mathcal{U}, \ell_M).2}$	<u>aiEnc</u> : same as in $\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}$ <u>aiDKGen</u> : $\mathcal{U}' = \mathcal{U}: (\hat{f}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f})$ where $\sum_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} = - \sum_{\text{pk} \in \mathcal{C} \cap \mathcal{U}} \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f \top} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}$	IND of UZS
$\mathbf{G}_{2.(\mathcal{U}, \ell_M).3}$	<u>aiEnc</u> : same as in $\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}$ <u>aiDKGen</u> : $\mathcal{U}' = \mathcal{U}: (\hat{f}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} + \Delta_{\text{pk}, \mathcal{U}}^{b, \ell_M, \ell_f})$ where $\sum_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} = - \sum_{\text{pk} \in \mathcal{C} \cap \mathcal{U}} \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f \top} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}$ $\Delta_{\text{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} = \hat{f}_{\text{pk}}(\hat{x}_{\text{pk}}^0) - \hat{f}_{\text{pk}}(\hat{x}_{\text{pk}}^b)$	Statistical
$\mathbf{G}_{2.(\mathcal{U}, \ell_M).4}$	<u>aiEnc</u> : same as in $\mathbf{G}_{2.(\mathcal{U}, \ell_M).1}$ <u>aiDKGen</u> : $\mathcal{U}' = \mathcal{U}: (\hat{f}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f \top} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} + \Delta_{\text{pk}, \mathcal{U}}^{b, \ell_M, \ell_f})$ where $\Delta_{\text{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} = \hat{f}_{\text{pk}}(\hat{x}_{\text{pk}}^0) - \hat{f}_{\text{pk}}(\hat{x}_{\text{pk}}^b)$	IND of UZS
$\mathbf{G}_{2.(\mathcal{U}, \ell_M)+1.0} :=$ $\mathbf{G}_{2.(\mathcal{U}, \ell_M).5}$	<u>aiEnc</u> : $(\mathcal{U}', \ell'_M) = (\mathcal{U}, \ell_M): (\hat{x}_{\text{pk}}^0, \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}, 0)$ <u>aiKeyGen</u> : $\mathcal{U}' = \mathcal{U}: (\hat{f}_{\text{pk}}, \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, 0)$	IND of AWIPE

Fig. 14. Intermediate hybrids for the transition from $\mathbf{G}_2$ to $\mathbf{G}_3$ in Theorem 3. We denote by $(\mathcal{U}', \ell'_M) < (\mathcal{U}, \ell_M)$ when $(\mathcal{U}', \ell'_M)$ is a previous pair of $(\mathcal{U}, \ell_M)$ in the encryption-query set $\mathcal{Q}_M$.

- For each complete $\text{QEnc}(\text{pk}, \hat{x}_{\text{pk}}^0, \hat{x}_{\text{pk}}^1, \mathcal{U}, \ell_M)$ query, $\mathcal{B}$ prepares the AWIPE message as

$$\begin{aligned} m^0 &= (\hat{x}_{\text{pk}}^b, \llbracket \mathbf{b}_{\text{pk}, \mathcal{U}, \ell_M}, 0 \rrbracket_1), \\ m^1 &= (\hat{x}_{\text{pk}}^b, \llbracket \mathbf{0}^{|\mathcal{U}|-1}, 1 \rrbracket_1); \end{aligned}$$

and sends (pk, m^0, m^1) to the AWIPE encryption oracle. It uses the returned ciphertext to complete the reply to $\mathcal{A}$.

- $\mathcal{B}$ outputs $\mathcal{A}$'s guess for the challenge bit $\text{AWIPE}.b$.

Let F_{AWIPE} be the functionality defined in Definition 5 for AWIPE. The admissibility condition $(*)$ of AWIPE in each transition holds since one always has

$$F_{\text{AWIPE}}(k^0, m^0) = \llbracket \hat{f}_{\text{pk}}(\hat{x}_{\text{pk}}^b) + \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} \rrbracket_T = F_{\text{AWIPE}}(k^1, m^1).$$

Therefore, in each transition of the multiple hybrid games for each $\text{pk} \in \mathcal{H}$, when $\text{AWIPE}.b = 0$, $\mathcal{A}$ is playing the previous game; when $\text{AWIPE}.b = 1$, $\mathcal{A}$ is playing the subsequent game. Since the number of $\text{pk} \in \mathcal{H}$ queried to either QEnc or QDKGen is bounded by $(q_{\text{QNewHon}} - q_{\text{QCor}})$, one has

$$\left| \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 0}} - \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 1}} \right| \leq (q_{\text{QNewHon}} - q_{\text{QCor}}) \cdot \text{Adv}_{\text{AWIPE}}^{\text{sel-sym}}.$$

□

Lemma 10 (AWS-DDFE: Transition from $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 1}$ to $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}$). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is*

$$\left| \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 1}} - \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}} \right| \leq \text{Adv}_{\text{UZS}}^{\text{otu-sta}}.$$

Proof. We build an adversary $\mathcal{B}$ against the otu-sta-IND security of UZS from an adversary $\mathcal{A}$ that distinguishes between two games in the transition. To simulate a AWS-DDFE challenger, $\mathcal{B}$ uses the UZS oracles to handle all UZS related operations.

- For each complete $\text{QEnc}(\text{pk}, \hat{x}_{\text{pk}}^0, \hat{x}_{\text{pk}}^1, \mathcal{U}', \ell'_M)$ query, $\mathcal{B}$ prepares the AWIPE message as
 - If $(\mathcal{U}', \ell'_M) \neq (\mathcal{U}, \ell_M)$: it obtains $\mathbf{b}_{\text{pk}, \mathcal{U}', \ell'_M} \leftarrow \text{QTokGen}(\text{pk}, \mathcal{U}', \ell'_M)$ to complete m .
 - If $(\mathcal{U}', \ell'_M) = (\mathcal{U}, \ell_M)$: it does not have to obtain $\mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M}$ as the message to AWIPE encryption is $(\hat{x}^b, \llbracket \mathbf{0}^{|\mathcal{U}|-1}, 1 \rrbracket_1)$ in this case.
- For each complete $\text{QDKGen}(\text{pk}, (\hat{f}_{\text{pk}})_{\text{pk} \in \mathcal{U}}, \mathcal{U})$ query, $\mathcal{B}$ prepares the AWIPE key as
 - $\mathcal{B}$ obtains $\llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f} \rrbracket_2 \leftarrow \text{QSeedGen}(\text{pk}, \mathcal{U}, \ell_f)$;
 - $\mathcal{B}$ obtains $(\llbracket R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} \rrbracket_2)_{\text{pk} \in \mathcal{U} \cap \mathcal{H}} \leftarrow \text{QShare}(\mathcal{U}, \ell_f || \ell_M)$;
 where $\mathbf{f} = (f_{\text{pk}})_{\text{pk} \in \mathcal{U}}$ and completes the key with the inner-product input as

$$k = (\hat{f}_{\text{pk}}, \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f}, R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} \rrbracket_2).$$

The admissibility condition $(*)$ of UZS holds since

- all the corruption queries in AWS-DDFE are sent in one shot;
- $\text{QShare}(\mathcal{U}, \ell_f || \ell_M)$ queries are made for the same ℓ_M on every $\mathcal{U}$ and there are no $\text{QTokGen}(\text{pk}, \mathcal{U}, \ell_M)$ queries required.

When $\text{UZS}.b = 0$, one has $\llbracket R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} \rrbracket_2 = \llbracket \mathbf{a}_{\text{pk}, \mathcal{U}}^{\ell_f} \cdot \mathbf{b}_{\text{pk}, \mathcal{U}}^{\ell_M} \rrbracket_2$ which corresponds to $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 1}$; and when $\text{UZS}.b = 1$, one has $(\llbracket R_{\text{pk}, \mathcal{U}}^{\ell_M, \ell_f} \rrbracket_2)_{\text{pk} \in \mathcal{H} \cap \mathcal{U}} \xleftarrow{\$} \mathcal{R}_{\mathcal{H} \cap \mathcal{U}, \ell_f || \ell_M}$ (as in Definition 7), which corresponds to $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}$. Therefore, one has

$$\left| \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 1}} - \text{Adv}_{\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}} \right| \leq \text{Adv}_{\text{UZS}}^{\text{otu-sta}}.$$

□

Lemma 11 (AWS-DDFE: Transition from $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}$ to $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 3}$). *The two games $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 2}$ to $\mathbf{G}_{2, (\mathcal{U}, \ell_M), 3}$ are identical.*

Proof. By the fact that $\sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}} \Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} = 0$ (Remark 7), for any random shares $(R_{\mathbf{pk}, \mathcal{U}}^{\ell_M, \ell_f})_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}}$ of the relation

$$\sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}} R_{\mathbf{pk}, \mathcal{U}}^{\ell_M, \ell_f} = - \sum_{\mathbf{pk} \in \mathcal{C} \cap \mathcal{U}} \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}{}^\top \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M},$$

one has $(R_{\mathbf{pk}, \mathcal{U}}^{\ell_M, \ell_f} + \Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f})_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}}$ are also random shares of the same relation. $\square$

Lemma 12 (AWS-DDFE: Transition from $\mathbf{G}_{2.(\mathcal{U}, \ell_M).4}$ to $\mathbf{G}_{2.(\mathcal{U}, \ell_M).5}$). *For any PPT adversary $\mathcal{A}$, the advantage in distinguishing two games is*

$$\left| \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).4}} - \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).5}} \right| \leq (q_{\text{QNewHon}} - q_{\text{QCor}}) \cdot \text{Adv}_{\text{AWIPE}}^{\text{sel-sym}}.$$

Proof. Given $\{(\mathbf{pk}, \hat{x}_{\mathbf{pk}}^{\tau, 0}, \hat{x}_{\mathbf{pk}}^{\tau, 1}, \mathcal{U}, \ell_M)\}_{\tau \in [\text{rep}_{\ell_M}]}$ as a set of complete encryption queries and $(\mathbf{pk}, (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}, \mathcal{U})$ as a complete decryption-key query, one has the following facts by the Remark 7:

1. $\Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} = \hat{f}_{\mathbf{pk}}(\hat{x}_{\mathbf{pk}}^{0, \tau}) - \hat{f}_{\mathbf{pk}}(\hat{x}_{\mathbf{pk}}^{b, \tau}) \quad \forall \tau \in [\text{rep}_{\ell_M}]$,
2. $\sum_{\mathbf{pk} \in \mathcal{H} \cap \mathcal{U}} \Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} = 0$.

We proceed by using multiple hybrid games for each $\mathbf{pk} \in \mathcal{H}$. We build an adversary $\mathcal{B}$ against the sel-sym security of AWIPE from an adversary $\mathcal{A}$ that distinguishes between two games in the transition. To simulate a AWS-DDFE challenger, $\mathcal{B}$ uses the AWIPE oracles to handle all AWIPE related operations for the reply of each $(\mathbf{pk}, \mathcal{U})$ -involved query from $\mathcal{A}$.

- For each complete QDKGen $(\mathbf{pk}, (\hat{f}_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}, \mathcal{U})$ query, $\mathcal{B}$ prepares the AWIPE key as

$$\begin{aligned} k^0 &= (\hat{f}_{\mathbf{pk}}, \llbracket \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}, \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}{}^\top \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M} + \Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f} \rrbracket_2) \\ k^1 &= (\hat{f}_{\mathbf{pk}}, \llbracket \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}, \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}{}^\top \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M} \rrbracket_2) \end{aligned}$$

where $\mathbf{f} = (f_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}$ and sends $(\mathbf{pk}, k^0, k^1)$ to the AWIPE decryption key generation oracle. It uses the returned decryption key to complete the reply to $\mathcal{A}$.

- For each complete QEnc $(\mathbf{pk}, \hat{x}_{\mathbf{pk}}^0, \hat{x}_{\mathbf{pk}}^1, \mathcal{U}, \ell_M)$ query, $\mathcal{B}$ prepares the AWIPE message as

$$\begin{aligned} m^0 &= (\hat{x}_{\mathbf{pk}}^b, \llbracket \mathbf{0}^{|\mathcal{U}|-1}, 1 \rrbracket_1); \\ m^1 &= (\hat{x}_{\mathbf{pk}}^0, \llbracket \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M}, 0 \rrbracket_1); \end{aligned}$$

and sends $(\mathbf{pk}, \hat{m}^0, \hat{m}^1)$ to the AWIPE encryption oracle. It uses the returned ciphertext to complete the reply to $\mathcal{A}$.

- $\mathcal{B}$ outputs $\mathcal{A}$'s guess for the challenge bit AWIPE. b .

Let F_{AWIPE} be the functionality defined in Definition 5 for AWIPE. The admissibility condition (*) of AWIPE in each transition holds since one always has

$$\begin{aligned} F_{\text{AWIPE}}(k^0, m^0) &= \llbracket (\hat{f}_{\mathbf{pk}}(\hat{x}_{\mathbf{pk}}^b) + \Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f}) + \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}{}^\top \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M} \rrbracket_T \\ &= \llbracket \hat{f}_{\mathbf{pk}}(\hat{x}_{\mathbf{pk}}^0) + \mathbf{a}_{\mathbf{pk}, \mathcal{U}}^{\ell_f}{}^\top \cdot \mathbf{b}_{\mathbf{pk}, \mathcal{U}}^{\ell_M} \rrbracket_T \text{ (by the above fact 1)} \\ &= F_{\text{AWIPE}}(k^1, m^1) \end{aligned}$$

The index τ is omitted in the above equalities as $\Delta_{\mathbf{pk}, \mathcal{U}}^{b, \ell_M, \ell_f}$ applies to all pairs of $\tau \in [\text{rep}_{\ell_M}]$ and $\mathbf{f} = (f_{\mathbf{pk}})_{\mathbf{pk} \in \mathcal{U}}$. Therefore, in each transition of the multiple hybrid games for each $\mathbf{pk} \in \mathcal{H}$, when AWIPE. $b = 0$, $\mathcal{A}$ is playing the previous game; when AWIPE. $b = 1$, $\mathcal{A}$ is playing the subsequent game. Since the number of pairs $\mathbf{pk} \in \mathcal{H}$ queried to either QEnc or QDKGen is bounded by $(q_{\text{QNewHon}} - q_{\text{QCor}})$, one has

$$\left| \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).4}} - \text{Adv}_{\mathbf{G}_{2.(\mathcal{U}, \ell_M).5}} \right| \leq (q_{\text{QNewHon}} - q_{\text{QCor}}) \cdot \text{Adv}_{\text{AWIPE}}^{\text{sel-sym}}.$$

$\square$